

UNIVERSITÀ DEGLI STUDI DELL'INSUBRIA

DIPARTIMENTO DI SCIENZE TEORICHE E APPLICATE

CORSO DI STUDIO TRIENNALE IN
INFORMATICA

Sviluppo in Flutter di un'applicazione mobile basata su servizi di geolocalizzazione per la gestione personalizzata e la ricerca di centri dermatologici

Relatore:

Prof. Ignazio Gallo

Tesi di Laurea di:

Gabriele Caretti

Matricola 756564

Correlatori:

Andrea Carugno, Mattia Gatti

Anno accademico:

2025/2026

Indice

1 Introduzione **1**

1.1 Contesto 1

1.2 Obiettivi del progetto 2

1.3 Struttura della tesi 3

2 Stato dell'arte **5**

2.1 Inquadramento clinico 5

2.1.1 Dermatite atopica e psoriasi: caratteristiche e differenze . 5

2.1.2 Il percorso di cura e il ruolo dei centri specialistici 6

2.2 Strumenti esistenti per la ricerca di centri dermatologici 6

2.2.1 I portali dermatopia e vicini di pelle 6

2.2.2 Limiti degli strumenti attuali 7

2.3 Provider di mappe per applicazioni mobili 7

2.3.1 Geocodifica e ricerca per indirizzo 9

2.4 Framework per lo sviluppo cross-platform 10

3 Analisi dei requisiti **11**

3.1 Attori e destinatari del sistema 11

3.2 Requisiti funzionali 12

3.3 Requisiti non funzionali 15

3.4 Casi d'uso 17

4 Progettazione **24**

4.1 Architettura del sistema 24

4.1.1 Scelta tecnologica: il framework Flutter e Dart 26

4.1.2 Pattern architetturale e gestione stato 27

4.2 Progettazione dell'interfaccia 29

4.2.1 Flusso di navigazione e mappa dell'App 29

4.2.2 Progettazione del modulo mappa e filtri 30

4.3 Modello e persistenza dei dati 32

4.3.1 Rappresentazione e normalizzazione 32

4.3.2 Strategia di caching e funzionamento offline 34

5 Sviluppo **37**

5.1 Ambiente di sviluppo e librerie utilizzate 37

5.2 Acquisizione e normalizzazione dei dati 39

5.2.1 Interrogazione delle fonti e fallback 39

5.2.2 Normalizzazione dei record 41

5.2.3 Salvataggio sul dispositivo 43

5.3 Caricamento mappa e gestione dei marker 44

5.3.1 Livelli, marker e controlli della mappa 44

5.3.2 Algoritmo di clustering dei marker 47

5.4 Localizzazione e gestione dei permessi 50

5.5 Ricerca per indirizzo e geocodifica 52

5.6 Filtri e ricerca per prossimità 53

5.6.1 Distanze geodetiche e filtro per raggio 53

5.6.2 Individuazione del centro più vicino 55

5.7 Inserimento dei centri personalizzati 56

5.8 Integrazione con i servizi del dispositivo 58

5.9 Il modulo in esecuzione 60

6 Testing e validazione **62**

6.1 Strategia di testing 62

6.2 Unit test sulla logica applicativa 62

6.3 Verifiche manuali e compatibilità 64

7 Conclusioni **66**

7.1 Risultati raggiunti 66

7.2 Criticità e limiti attuali 67

7.3 Sviluppi futuri 67

7.3.1 Un livello intermedio di normalizzazione 67

7.3.2 Sincronizzazione fra dispositivi 68

7.3.3 Percorsi e mappe non raster vettoriali 68

Bibliografia **70**

Sitografia **71**

Capitolo 1

Introduzione

1.1 Contesto

La dermatite atopica e la psoriasi condividono una caratteristica che, dal punto di vista generale, pesa più di quanto si potrebbe pensare: non guariscono. Sono patologie infiammatorie croniche recidivanti, in cui fasi di remissione e fasi di riacutizzazione si alternano nel corso di anni o di decenni [5, 1]. Chi ne è affetto non ha bisogno di una visita isolata, ma di un percorso: controlli periodici, aggiustamenti della terapia topica o sistemica e, in una parte dei casi, trattamenti biologici che possono essere prescritti soltanto da strutture autorizzate.

L'Organizzazione Mondiale della Sanità (*OMS*), nel rapporto globale dedicato alla psoriasi, ha riconosciuto esplicitamente il ritardo diagnostico e la difficoltà di accesso alle cure specialistiche come problemi di salute pubblica, non come semplici disagi organizzativi [6]. In Italia il Piano Nazionale della Cronicità (*PNC*) muove nella stessa direzione, quando individua nella continuità assistenziale e nella presa in carico territoriale i due assi su cui intervenire [2].

Tradotto in una domanda concreta, quella che un paziente si pone davvero: dove si trova il centro più vicino che segue la mia patologia, e come ci arrivo? La risposta esiste, ma è distribuita. Le strutture di riferimento sono censite, però l'informazione vive in elenchi separati, ciascuno con la propria organizzazione e i propri criteri di aggiornamento. Due portali in particolare raccolgono i centri italiani dedicati alla dermatite atopica e alla psoriasi [11, 29], e sono un buon punto di partenza; restano tuttavia pagine web pensate per essere lette su un monitor, dove la ricerca procede per regione o per provincia e la distanza dall'utente non è una grandezza che il sistema conosca o di cui tenga conto.

Uno smartphone, al contrario, dispone già di queste informazioni: conosce la propria posizione, è in grado di visualizzare una mappa, di avviare la navigazione, di effettuare una chiamata e di conservare dati anche in assenza di connessione. Da questa asimmetria nasce il lavoro descritto nelle pagine seguenti: un'applicazione mobile che rende interrogabili per posizione dati già pubblici, ossia nella stessa forma in cui la domanda viene formulata.

1.2 Obiettivi del progetto

L'obiettivo generale è la progettazione e la realizzazione di un'applicazione mobile cross-platform, sviluppata con il framework Flutter, che permetta di individuare su mappa i centri dermatologici italiani dedicati alla dermatite atopica e alla psoriasi, e di integrarli con voci inserite dall'utente. In termini più operativi, il progetto si è dato i seguenti obiettivi:

- Acquisire l'elenco dei centri da sorgenti pubbliche già esistenti, senza richiedere la costruzione di un nuovo archivio proprietario e senza duplicare un lavoro di censimento già svolto da altri;
- Normalizzare dati provenienti da servizi con schemi diversi in un'unica rappresentazione interna coerente, gestendo i casi di record incompleti o malformati;
- Visualizzare i centri su una mappa interattiva, con un clustering automatico dei marker che mantenga la scena leggibile anche quando le strutture sono concentrate nella stessa area urbana;
- Consentire il filtraggio per patologia e per raggio di distanza dalla posizione dell'utente, con l'ordinamento dei risultati per distanza geodetica;
- Offrire una ricerca per indirizzo o per località, appoggiandosi a un servizio di geocodifica;
- Permettere all'utente di salvare i centri di interesse tra i preferiti e di inserire, modificare ed eliminare centri propri, conservati localmente sul dispositivo;
- Integrare le azioni che seguono la consultazione, cioè l'avvio della navigazione verso il centro, la chiamata telefonica, l'invio di un'email e la condivisione della scheda;

- Garantire un funzionamento accettabile in assenza di connessione, attraverso una cache persistente dell'ultimo elenco scaricato.

È altrettanto importante definire ciò che il progetto non si propone di fare.L'applicazione non formula diagnosi e non esprime valutazioni cliniche, si limita a mostrare dove si trovano le strutture e a fornirne i contatti. Non prevede autenticazione né profili utente e, di conseguenza, non trasmette all'esterno alcun dato personale. Non include un backend dedicato, e si basa su servizi pubblicamente disponibili che agiscono da fonti.

1.3 Struttura della tesi

Il capitolo 2 inquadra il problema, prima dal lato clinico e poi dal lato tecnologico. Esamina gli strumenti già disponibili per cercare un centro dermatologico e ne individua i limiti, confronta le piattaforme cartografiche utilizzabili in ambito *mobile* e motiva la scelta del framework di sviluppo.

Il capitolo 3 formalizza attori, requisiti funzionali e requisiti non funzionali, e li raccoglie in un diagramma dei casi d'uso seguito dalle schede dei tre scenari principali.

Il capitolo 4 descrive l'architettura a livelli dell'App, la strategia di gestione dello stato, l'impianto dell'interfaccia (*UI e UX*) e il modello dei dati adottato per riconciliare sorgenti eterogenee.

Il capitolo 5 ripercorre l'implementazione dell'App. L'acquisizione dei dati con endpoint di riserva, normalizzazione, persistenza, caricamento mappa, algoritmo di clustering dei marker, macchina a stati, permessi di localizzazione, ricerca per indirizzo, filtri e integrazione con i servizi esterni disponibili sul dispositivo.

Il capitolo 6 riporta la strategia di verifica e i risultati ottenuti.

Il capitolo 7 raggruppa l'intero operato, elenca i limiti che restano aperti e indica le possibili direzioni di sviluppo future.

Ogni capitolo prepara la basi per il successivo, come mostrato nella figura 1.1:

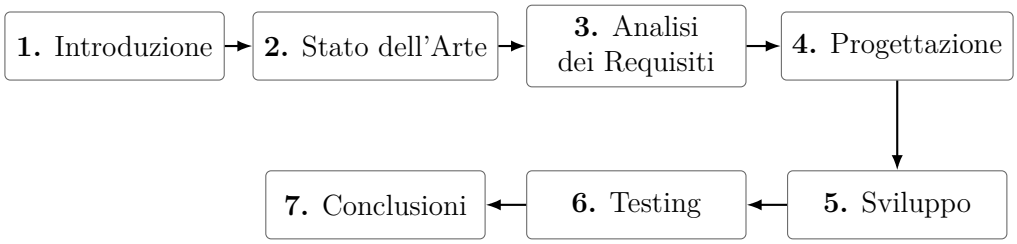


Figura 1.1: Struttura della tesi.

Capitolo 2

Stato dell'arte

2.1 Inquadramento clinico

2.1.1 Dermatite atopica e psoriasi: caratteristiche e differenze

Un'applicazione che filtra i centri per patologia deve sapere perché quel filtro serve. Dermatite atopica e psoriasi vengono spesso accostate, e in effetti hanno in comune la cronicità, la componente infiammatoria e un impatto sulla qualità della vita che va oltre il sintomo cutaneo; i meccanismi che le sostengono, però, sono distinti, e distinti sono i percorsi terapeutici.

La dermatite atopica è caratterizzata da un difetto della barriera cutanea associato a una risposta immunitaria di tipo 2. Esordisce tipicamente nell'infanzia, si manifesta con lesioni eczematose e prurito intenso, e ha un andamento fortemente stagionale [5]. La psoriasi ha un profilo immunologico differente, dominato dall'asse interleuchina-23 e dei linfociti T helper 17, si presenta con placche eritematose a desquamazione argentea e, in una quota rilevante di pazienti, si accompagna a manifestazioni articolari e a comorbilità di tipo metabolico e cardiovascolare [1].

La conseguenza organizzativa è che le due patologie non vengono seguite necessariamente dalle stesse strutture. Alcuni ambulatori sono dedicati a una sola delle due, altri a entrambe, e la specializzazione si riflette sui farmaci disponibili e sui protocolli di monitoraggio. Un elenco che non distingua i due casi costringe l'utente a verificare struttura per struttura, che è esattamente il lavoro che si vorrebbe evitargli. Per questo, nel modello dei dati descritto al paragrafo 4.3.1, la patologia trattata è un attributo di prima classe del centro e non un'annotazione testuale.

2.1.2 Il percorso di cura e il ruolo dei centri specialistici

Nelle forme moderate e severe la gestione esce dall'ambito della medicina generale. I farmaci biologici e le piccole molecole di più recente introduzione richiedono una prescrizione specialistica, esami di monitoraggio periodici e, in Italia, l'erogazione attraverso centri individuati a livello regionale. Questo genera una geografia particolare: le strutture non sono distribuite in modo uniforme sul territorio, e la distanza dal centro più vicino diventa un fattore che incide sull'aderenza alla terapia.

Il Piano Nazionale della Cronicità (*PNC*) individua nella frammentazione informativa uno degli ostacoli alla presa in carico [2]. È una frammentazione che l'utente percepisce in modo molto concreto, sotto forma di elenchi da confrontare a mano. Ridurre quel confronto a un'unica vista ordinata per distanza è l'obiettivo minimo che un'applicazione come questa deve raggiungere per essere utile.

2.2 Strumenti esistenti per la ricerca di centri dermatologici

2.2.1 I portali dermatopia e vicini di pelle

Nel panorama italiano due iniziative si distinguono per completezza. La prima, Dermatopia, espone un elenco di centri dermatologici attraverso un servizio web pubblico [11]. La seconda, Vicini di Pelle, è un progetto divulgativo dedicato alla psoriasi che pubblica una mappa delle strutture di riferimento, costruita sopra un plugin cartografico per WordPress [29].

Entrambe risolvono la parte difficile del problema, cioè il censimento. Raccogliere e mantenere aggiornato un elenco di strutture sanitarie è un lavoro redazionale continuo, che nessuna tesi triennale potrebbe replicare con la stessa affidabilità. Per questa ragione il progetto non tenta di costruire un archivio proprio: assume queste sorgenti come dato di partenza e concentra il proprio contributo su ciò che manca a valle, cioè l'accesso mobile e georeferenziato.

2.2.2 Limiti degli strumenti attuali

Dall'analisi dei due portali emergono alcuni limiti ricorrenti, che hanno orientato in modo diretto i requisiti del capitolo 3.

La navigazione è organizzata per suddivisione amministrativa e non per prossimità. L'utente sceglie una regione o una provincia e ottiene un elenco; se abita al confine tra due province, deve ripetere la ricerca. La domanda originaria, cioè quale sia il centro più vicino, non ha una risposta diretta.

La posizione dell'utente non viene utilizzata. Nessuno dei due strumenti chiede o impiega le coordinate del dispositivo, quindi non esiste ordinamento per distanza né filtro per raggio.

L'esperienza sui piccoli schermi è penalizzata. Le tabelle a più colonne richiedono lo scorrimento orizzontale, e le azioni successive alla consultazione, come chiamare la struttura o avviare la navigazione, richiedono la copia manuale del dato.

Non esiste alcun funzionamento offline. Ogni consultazione richiede una connessione attiva, condizione non garantita quando la ricerca avviene in viaggio.

Infine, non è previsto alcun contributo da parte dell'utente. Se il proprio dermatologo di riferimento non compare nell'elenco, non c'è modo di annotarlo e ritrovarlo insieme agli altri. Questa osservazione ha portato alla funzionalità dei centri personalizzati, che nella versione realizzata convivono con quelli remoti nella stessa mappa e negli stessi filtri.

2.3 Provider di mappe per applicazioni mobili

La scelta del provider delle mappe condiziona costi, vincoli di licenza e struttura del codice, quindi va affrontata prima di scrivere l'applicazione. Sono state valutate tre alternative.

1. Google Maps Platform, integrabile in Flutter attraverso un plugin ufficiale. Offre la qualità cartografica più alta e servizi accessori maturi, ma richiede una chiave API associata a un profilo di fatturazione e adotta un modello

a consumo, con una soglia mensile gratuita oltre la quale ogni caricamento di mappa viene tariffato [19]. Per un'applicazione senza modello di ricavo, il rischio è di trovarsi con un costo che cresce insieme al numero di utenti.

2. Mapbox, che propone cartografia vettoriale personalizzabile e un modello di prezzo analogo, basato su un numero di caricamenti mensili gratuiti e su tariffe successive [23]. La qualità dello stile è notevole, ma restano la dipendenza da un provider commerciale e la necessità di un token.

3. OpenStreetMap, i cui dati sono distribuiti con licenza Open Database License e richiedono soltanto l'attribuzione [25]. In Flutter il compito è affidato a `flutter_map`, un package mantenuto dalla *community* che effettua il rendering dei *tile layer* (i tasselli quadrati che compongono la mappa) senza legarsi a un provider specifico [13, 12]. Non sono necessari né una chiave né un profilo di fatturazione e la sorgente dei *tile* può essere sostituita modificando un URL.

Criterio	**Google Maps**	**Mapbox**	**OSM con flutter_map**
Costi operativi	A consumo oltre la soglia gratuita	A consumo oltre la soglia gratuita	Nessun costo diretto
Credenziali richieste	Chiave API con fatturazione attiva	Token di accesso	nessuna
Licenza	Proprietaria	Proprietaria	ODbL con attribuzione
Marker personalizzabili	Vincolata al plugin nativo	Buona	Totale, sono widget Flutter
Sostituibilità del provider	Bassa	Bassa	Alta
Vincoli d'uso	Contrattuali	Contrattuali	Policy di utilizzo dei tile

Tabella 2.1: Confronto dei provider delle mappe valutati.

A seguito di un'attenta valutazione, la scelta migliore è risultata OpenStreetMap con `flutter_map`. I fattori determinanti non sono stati solamente i costi, ma anche il fatto che i marker siano widget Flutter a tutti gli effetti: questo permette di costruire il clustering dinamico descritto al paragrafo 5.3.2 con gli stessi strumenti usati per il resto dell'interfaccia, senza passare attraverso un layer nativo.

Tuttavia, esiste un compromesso da questa scelta. I *tile* pubblici di OpenStreetMap sono forniti a titolo gratuito e la relativa policy vieta usi massivi, impone di identificare l'applicazione richiedente e richiede di rispettare le direttive di caching [27]: sono vincoli che vanno rispettati e che andrebbero gestiti, specialmente nel caso in cui il numero di utenti cresca a numeri considerevoli. In secondo luogo, la grafica dei *tile* raster è meno rifinita di quella vettoriale offerta dai concorrenti commerciali.

Sul piano tecnico la visualizzazione segue lo schema *slippy map*: la superficie terrestre è proiettata secondo Web Mercator (standard per le mappe digitiali web) e suddivisa, a ogni livello di zoom, in una griglia di immagini quadrate indicizzate da tre valori, cioè zoom, colonna e riga [28]. Ogni incremento dello zoom raddoppia il numero di tile per lato, come illustrato nella figura 2.1. Questa relazione esponenziale viene trattata anche nella formula che dimensiona le celle dell'algoritmo di clustering.

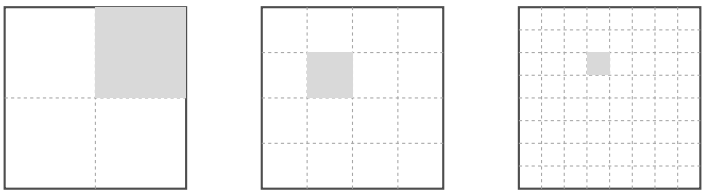


Figura 2.1: Schema slippy map: la stessa porzione di territorio ai diversi livelli di zoom, con un tile evidenziato per ciascuno. Ogni incremento dello zoom raddoppia il numero di tile per lato e ogni tile è individuato dagli indici di zoom, colonna e riga.

2.3.1 Geocodifica e ricerca per indirizzo

Una mappa senza barra di ricerca è utile soltanto a chi sa già dove guardare. Serve quindi una geocodifica diretta, cioè la traduzione di un testo libero in coordinate, come una barra di ricerca per l'indirizzo. Nell'ecosistema OpenStreetMap questo servizio è offerto da Nominatim, che espone un endpoint di ricerca con risposta in formato JSON.

Anche qui valgono vincoli precisi, dichiarati nella policy di utilizzo: al massimo una richiesta al secondo, obbligo di identificare l'applicazione, divieto di geocodifica massiva e invito esplicito a memorizzare i risultati (caching) per non ripetere interrogazioni identiche [26]. La soluzione adottata, ossia un meccanismo di *debounce* sull'input descritto al paragrafo 5.5, nasce proprio da questo vincolo.

2.4 Framework per lo sviluppo cross-platform

Il requisito di una singola base di codice per Android e iOS è molto importante. Lo sviluppo nativo separato, con Kotlin e Swift, resta la soluzione con il migliore accesso alle API di sistema, ma comporta due implementazioni parallele dell'interfaccia, serve quindi una soluzione alternativa.

React Native adotta un approccio a componenti pilotati da JavaScript [24], ed è una buona scelta quando si dispone già di competenze nell'ecosistema web.

Infine, Flutter segue una strada diversa: compila codice Dart in binario e disegna l'interfaccia con un proprio motore grafico, ottenendo un controllo completo sul risultato visivo e un comportamento uniforme sulle due piattaforme [18, 17].

Per questo progetto quindi, Flutter presentava due vantaggi principali.

1. Il primo è il controllo totale sul rendering, indispensabile quando i marker della mappa devono cambiare forma e dimensione in funzione del livello di zoom.

2. Il secondo è la disponibilità di Material 3 come design system già implementato [20], che ha permesso di ottenere un'interfaccia coerente senza costruire da zero i componenti.

A questi si aggiunge un ecosistema di package che copre già le esigenze del progetto, dalla geolocalizzazione alla persistenza dei dati in locale, elencati nel dettaglio al paragrafo 5.1.

Capitolo 3

Analisi dei requisiti

3.1 Attori e destinatari del sistema

Il destinatario dell'applicazione è una sola figura, e conviene descriverla con precisione perché da lei dipendono le scelte dell'interfaccia. Si tratta di una persona, per esempio colui che è affetto da dermatite atopica o psoriasi, ma potrebbe anche essere un suo familiare o un dottore, che deve individuare una struttura specializzata raggiungibile. Non ha competenze informatiche particolari, usa l'applicazione su uno smartphone e spesso con una sola mano. Non è necessariamente un operatore sanitario, non inserisce dati clinici e non ha bisogno di funzioni di refertazione.

Da questa analisi derivano due conseguenze operative:

1. Nessuna funzione può richiedere una registrazione, perché ogni passaggio in più tra l'apertura dell'applicazione e la risposta alla domanda *dove è il centro più vicino* è un passaggio che riduce l'utilizzabilità dello strumento.

2. La seconda è che i controlli devono essere collocati nella metà inferiore dello schermo, dov'è possibile arrivarci più facilmente con il pollice.

Accanto all'attore primario operano diversi sistemi esterni, che nella terminologia UML (*Unified Modeling Language*) sono attori secondari. Questi non hanno intenzioni, ma il sistema dipende da loro e deve prevedere il caso in cui non rispondano.

Attore	Tipo	Ruolo nel sistema
Utente	Primario	Consulta la mappa, applica i filtri, gestisce preferiti e centri personalizzati, avvia le azioni di contatto e navigazione.
Servizio elenco centri	Secondario	Fornisce l'elenco dei centri dermatologici tramite endpoint HTTP pubblici. Il sistema ne interroga due, uno principale e uno secondario di riserva.
Servizio di geocodifica	Secondario	Traduce il testo digitato dall'utente in coordinate geografiche.
Server dei tile	Secondario	Fornisce le immagini che compongono la mappa.
Sistema operativo	Secondario	Espone la posizione del dispositivo e fa da tramite per garantirne il relativo permesso.
Applicazioni esterne	Secondario	Ricevono le richieste di navigazione, chiamata, invio email e condivisione.

Tabella 3.1: Attori del sistema e rispettivo ruolo.

3.2 Requisiti funzionali

I requisiti funzionali sono raccolti nella tabella 3.2, raggruppati per area. La colonna di priorità distingue i requisiti obbligatori necessari per il funzionamento all'applicazione, e da quelli desiderabili che migliorano l'esperienza ma la cui assenza non ne compromette il funzionamento.

ID	Descrizione	Priorità
Acquisizione e persistenza dei dati		
RF-01	Il sistema deve scaricare l'elenco dei centri dermatologici da un servizio pubblico remoto.	Obbligatorio
RF-02	In caso di errore o di risposta non valida dal servizio principale, il sistema deve interrogare automaticamente un endpoint secondario.	Obbligatorio
RF-03	Il sistema deve normalizzare i record provenienti da sorgenti con schemi differenti in un'unica rappresentazione comune interna.	Obbligatorio
RF-04	Il sistema deve scartare i record privi di coordinate valide senza interromperne l'elaborazione dei restanti.	Obbligatorio
RF-05	Il sistema deve conservare l'ultimo elenco scaricato in una cache persistente e utilizzarla quando la rete è assente.	Obbligatorio

segue alla pagina successiva

ID	Descrizione	Priorità
Visualizzazione cartografica		
RF-06	Il sistema deve mostrare una mappa interattiva con supporto a spostamenti, zoom e rotazione.	Obbligatorio
RF-07	Il sistema deve riportare i crediti della mappa originale in modo visibile.	Desiderabile
RF-08	Ogni centro deve essere rappresentato da un marker posizionato sulle proprie coordinate.	Obbligatorio
RF-09	Marker geograficamente vicini devono essere *clusterizzati* (raggruppati) in un unico indicatore che riporti il numero di centri contenuti.	Obbligatorio
RF-10	La selezione di un *cluster* (gruppo) deve produrre un avvicinamento della vista che lo scomponga negli elementi che lo costituiscono.	Obbligatorio
RF-11	Il sistema deve disegnare soltanto i marker compresi nell'area visibile, con un margine di tolleranza.	Desiderabile
Localizzazione e ricerca		
RF-12	Il sistema deve richiedere il permesso di accesso alla posizione soltanto quando l'utente attiva una funzione che la richieda.	Obbligatorio
RF-13	Il sistema deve centrare la vista sulla posizione dell'utente e indicarla con un simbolo distinto da quello dei centri.	Obbligatorio
RF-14	In caso di permesso negato il sistema deve continuare a funzionare, utilizzando una posizione iniziale predefinita.	Obbligatorio
RF-15	Se il permesso è stato negato in modo permanente, il sistema deve offrire un collegamento alle impostazioni di sistema.	Desiderabile
RF-16	Il sistema deve permettere la ricerca di una località o di un indirizzo tramite un campo di input testuale.	Obbligatorio
RF-17	Durante la digitazione il sistema deve proporre suggerimenti, interrogando il servizio di geocodifica solo dopo una pausa nell'inserimento.	Obbligatorio
RF-18	La selezione di un suggerimento deve spostare la vista sulle coordinate corrispondenti.	Obbligatorio
Filtraggio e ricerca per prossimità		

segue alla pagina successiva

ID	Descrizione	Priorità
RF-19	Il sistema deve consentire di filtrare i centri per patologia trattata, distinguendo dermatite atopica e psoriasi.	Obbligatorio
RF-20	Il sistema deve consentire di limitare i risultati a un raggio configurabile intorno alla posizione dell'utente.	Obbligatorio
RF-21	Il sistema deve presentare l'elenco dei centri più vicini ordinato per distanza crescente, con la distanza espressa in chilometri.	Obbligatorio
RF-22	Il calcolo della distanza deve tenere conto della curvatura terrestre.	Obbligatorio
RF-23	I filtri per patologia e per raggio devono poter essere combinati e devono applicarsi anche ai centri inseriti dall'utente.	Obbligatorio
RF-24	Il sistema deve rappresentare graficamente sulla mappa il raggio di ricerca attivo.	Desiderabile
Centri personalizzati e preferiti		
RF-25	Il sistema deve permettere l'inserimento di un centro non presente nell'elenco, indicandone denominazione, indirizzo, città, contatti e patologie trattate.	Obbligatorio
RF-26	La posizione del centro inserito deve essere acquisita dalla mappa, mediante pressione prolungata sul punto desiderato oppure assumendo il centro della viewport corrente, senza digitazione manuale delle coordinate.	Obbligatorio
RF-27	Il sistema deve validare i dati inseriti prima del salvataggio, richiedendo nome, indirizzo e città, almeno un contatto tra telefono ed email, almeno una patologia, e verificando che il numero telefonico rispetti il formato nazionale e che l'indirizzo email sia sintatticamente corretto.	Obbligatorio
RF-28	Il sistema deve permettere la modifica e l'eliminazione dei centri inseriti dall'utente.	Obbligatorio
RF-29	I centri personalizzati devono persistere tra un avvio e l'altro dell'applicazione ed essere distinguibili sulla mappa da quelli remoti.	Obbligatorio
RF-30	Il sistema deve permettere di aggiungere un centro ai preferiti e di poter accedere all'elenco.	Obbligatorio

segue alla pagina successiva

ID	Descrizione	Priorità
Azioni sul centro selezionato		
RF-31	La selezione di un marker deve aprire una scheda con nome, indirizzo e contatti disponibili.	Obbligatorio
RF-32	Il sistema deve consentire di avviare la navigazione verso il centro delegandola a un'applicazione esterna.	Obbligatorio
RF-33	Il sistema deve permettere di scegliere il mezzo di trasporto per il calcolo del percorso.	Desiderabile
RF-34	Il sistema deve consentire di avviare una chiamata telefonica e di comporre un'email verso i contatti del centro, quando presenti.	Obbligatorio
RF-35	Il sistema deve consentire la condivisione della scheda del centro tramite applicazioni esterne.	Desiderabile
Controlli della mappa		
RF-36	Il sistema deve fornire comandi di avvicinamento (*zoom in*) e allontanamento (*zoom out*) della vista alternativi al gesto a due dita.	Desiderabile
RF-37	Quando la mappa è ruotata, il sistema deve mostrare una bussola che ne consenta il ripristino verso il nord.	Desiderabile

Tabella 3.2: Requisiti funzionali del sistema.

3.3 Requisiti non funzionali

I requisiti non funzionali riguardano il modo in cui il sistema deve comportarsi, piuttosto che delle funzioni che deve offrire. Nel caso specifico assumono un peso maggiore del solito per due ragioni: l'applicazione dipende da servizi di terzi che non controlla, e opera in un contesto d'uso mobile in cui la connettività non è garantita.

ID	Categoria	Descrizione
RNF-01	Usabilità	I comandi d'uso frequente devono trovarsi nella porzione inferiore dello schermo, raggiungibile con il pollice e una sola mano.
RNF-02	Usabilità	Ogni elemento interattivo deve avere un'area di tocco non inferiore a 48 punti logici (pt) per lato, in accordo con le indicazioni di Material 3.

segue alla pagina successiva

ID	Categoria	Descrizione
RNF-03	Usabilità	Ogni azione dell'utente deve produrre un feedback visivo immediato, così da rendere percepibile lo stato del sistema.
RNF-04	Usabilità	I messaggi rivolti all'utente devono essere in italiano e privi di terminologia tecnica o di codici di errore.
RNF-05	Prestazioni	Lo spostamento e lo zoom della mappa devono risultare fluidi anche con l'intero elenco dei centri caricato in memoria.
RNF-06	Prestazioni	Il ricalcolo dei cluster non deve essere eseguito a ogni fotogramma dell'interazione, ma solo al termine di essa.
RNF-07	Prestazioni	L'avvio dell'applicazione non deve attendere il completamento delle richieste di rete per mostrare la mappa.
RNF-08	Stabilità	Ogni richiesta di rete deve avere un *timeout* oltre il quale viene scartata, per non bloccare l'interfaccia.
RNF-09	Stabilità	Un record malformato proveniente dal servizio remoto non deve impedire il caricamento dei record corretti.
RNF-10	Stabilità	In assenza di connessione il sistema deve restare utilizzabile con i dati già acquisiti, segnalando all'utente la condizione.
RNF-11	Stabilità	Un errore di lettura della cache locale non deve impedire l'avvio dell'applicazione.
RNF-12	Compatibilità	L'attribuzione dei dati cartografici deve essere sempre visibile, come previsto dalla licenza della sorgente.
RNF-13	Compatibilità	Le richieste ai servizi pubblici devono identificare l'applicazione e rispettare i limiti di frequenza dichiarati nelle rispettive policy.
RNF-14	Riservatezza	La posizione dell'utente deve essere impiegata per il centraggio della mappa e per il calcolo delle distanze, e non deve essere trasmessa ad alcun servizio.
RNF-15	Riservatezza	Il sistema non deve richiedere registrazione, credenziali o dati personali identificativi.
RNF-16	Portabilità	L'applicazione deve essere realizzata con un'unica base di codice per Android e iOS, dichiarando su entrambe le piattaforme i permessi necessari.

segue alla pagina successiva

ID	Categoria	Descrizione
RNF-17	Manutenibilità	Il modello dei dati interno deve essere indipendente dallo schema delle sorgenti remote, così da consentire la sostituzione di un endpoint senza modifiche all'interfaccia.

Tabella 3.3: Requisiti non funzionali del sistema.

3.4 Casi d'uso

Il diagramma della figura 3.1 raccoglie le interazioni principali. L'utente compare a sinistra, i casi d'uso sono racchiusi nel confine, e i servizi esterni sono collegati con linee tratteggiate ai casi d'uso che li coinvolgono: la cartina geografica e l'acquisizione dell'elenco dipendono da servizi remoti, la ricerca per indirizzo dalla geocodifica, la ricerca dei centri vicini dal sistema operativo, mentre indicazioni e contatti delegano il compito ad applicazioni installate sul dispositivo.

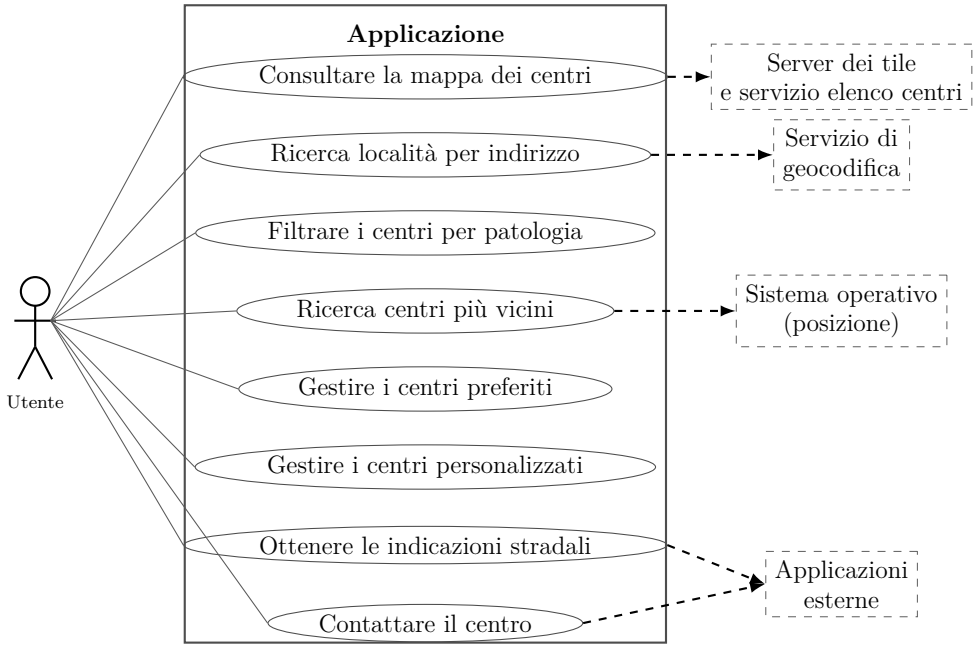


Figura 3.1: Diagramma dei casi d'uso.

I casi d'uso sono numerati da UC-01 a UC-08 seguendo l'ordine con cui compaiono nel diagramma, dall'alto verso il basso. Le schede che seguono ne mostrano nel dettaglio sei: sono esclusi il filtro per patologia e la gestione dei preferiti, il

cui svolgimento si riduce all'attivazione di un comando e all'aggiornamento della vista (*view*), senza azioni particolari. L'ultima riga di ogni scheda richiama i requisiti funzionali (RF) corrispondenti.

UC-01 — Consultare la mappa dei centri	
Attore primario	Utente
Attori secondari	Servizio elenco centri, server dei tile
Precondizioni	L'applicazione è stata avviata. Nessun permesso necessario.
Postcondizioni	La mappa mostra i centri disponibili nell'area visibile.
Scenario principale	1. L'utente apre l'applicazione. 2. Il sistema mostra la mappa su una viewport iniziale predefinita e carica i centri dalla cache locale, se presente. 3. Il sistema richiede l'elenco aggiornato al servizio remoto, normalizza i record e aggiorna la cache. 4. Il sistema seleziona i centri compresi nell'area visibile, li raggruppa in base al livello di zoom e disegna marker e cluster. 5. L'utente sposta la viewport oppure ne modifica lo zoom. 6. Il sistema ricalcola selezione e cluster al termine del gesto e aggiorna il disegno.
Scenari alternativi	3a. Il servizio principale non risponde oppure restituisce dati invalidi: il sistema interroga la fonte alternativa e prosegue dal passo 4. 3b. Nessuna fonte raggiungibile ma la cache contiene dati: il sistema utilizza la cache e segnala che l'elenco potrebbe non essere aggiornato. 3c. Nessuna fonte raggiungibile e cache vuota: il sistema segnala l'assenza di dati e offre un comando per riprovare. 4a. Alcuni record sono privi di coordinate valide: il sistema li scarta e disegna i rimanenti.
Requisiti	RF-01, RF-02, RF-03, RF-04, RF-05, RF-06, RF-07, RF-08, RF-09, RF-10, RF-11

Tabella 3.4: Scheda caso d'uso UC-01.

UC-02 — Ricerca località per indirizzo	
Attore primario	Utente
Attori secondari	Servizio di geocodifica
Precondizioni	La mappa è visualizzata ed è disponibile una connessione di rete.
Postcondizioni	La vista è centrata sulla località scelta.
Scenario principale	1. L'utente tocca il campo di ricerca e digita un indirizzo oppure il nome di una località. 2. Il sistema attende una pausa nella digitazione, quindi interroga il servizio di geocodifica con il testo inserito. 3. Il sistema mostra l'elenco dei risultati sotto il campo di ricerca. 4. L'utente seleziona un risultato. 5. Il sistema chiude l'elenco, sposta la vista sulle coordinate corrispondenti e ricalcola i marker visibili.
Scenari alternativi	2a. L'utente continua a digitare prima della scadenza del timeout (attesa): il sistema annulla la query in corso e ne pianifica una nuova, così da evitare multiple richieste per ogni carattere inserito. 3a. Il servizio non restituisce alcun risultato: il sistema segnala l'esito e lascia il testo inserito. 3b. Il servizio è irraggiungibile: il sistema segnala l'errore e mantiene la viewport corrente. 4a. L'utente annulla la ricerca: il sistema chiude l'elenco dei suggerimenti e ripristina i comandi della mappa.
Requisiti	RF-16, RF-17, RF-18

Tabella 3.5: Scheda caso d'uso UC-02.

UC-04 — Ricerca centri più vicini	
Attore primario	Utente
Attori secondari	Sistema operativo
Precondizioni	La mappa e l'elenco dei centri sono stati caricati.
Postcondizioni	L'utente dispone dell'elenco dei centri entro il raggio prescelto, ordinati per distanza.
Scenario principale	1. L'utente attiva la funzione di ricerca per raggio. 2. Il sistema verifica se il servizio di localizzazione del dispositivo sia attivo e se il permesso sia già stato concesso. 3. Il sistema richiede la posizione aggiornata al sistema operativo. 4. L'utente seleziona il raggio di ricerca e, se lo desidera, la patologia di interesse. 5. Il sistema calcola la distanza di ogni centro dalla posizione dell'utente, scarta quelli troppo distanti e applica il filtro per patologia sia ai centri remoti sia a quelli personalizzati 6. Il sistema disegna sulla mappa la circonferenza corrispondente al raggio e presenta l'elenco dei centri trovati ordinati per distanza crescente.
Scenari alternativi	2a. Il permesso non viene concesso: il sistema lo richiede e prosegue dal passo 3 se viene dato. 2b. Il permesso viene negato: il sistema mantiene la viewport predefinita, avvisa che la funzione richiede la posizione e chiede di riprovare più tardi. 2c. Il permesso viene negato in modo permanente: il sistema offre un collegamento alle impostazioni di sistema, poichè da questo stato la richiesta non può più essere ripresentata dall'applicazione. 2d. Il servizio di geolocalizzazione del dispositivo è disattivato: il sistema lo segnala e invita ad attivarlo. 6a. Nessun centro nel raggio: il sistema lo comunica e suggerisce di effettuare nuovamente la ricerca con un raggio più ampio.
Requisiti	RF-12, RF-13, RF-14, RF-15, RF-19, RF-20, RF-21, RF-22, RF-23, RF-24

Tabella 3.6: Scheda caso d'uso UC-04.

UC-06 — Inserimento centro personalizzato	
Attore primario	Utente
Attori secondari	Nessuno. L'operazione avviene interamente in locale e non richiede una connessione di rete.
Precondizioni	La mappa è aperta.
Postcondizioni	Il centro viene memorizzato sul dispositivo, ed è visibile sulla mappa come un marker.
Scenario principale	1. L'utente mantiene premuto il dito sulla mappa nel punto in cui si trova la struttura. 2. Il sistema apre il modulo di inserimento assumendo quel punto come posizione del centro. 3. L'utente compila nome, indirizzo e città, indica almeno un contatto tra numero di telefono e indirizzo email, e seleziona le patologie trattate. 4. L'utente conferma il salvataggio. 5. Il sistema verifica e valida i dati inseriti. 6. Il sistema assegna al centro un identificatore, lo aggiunge all'archivio locale e lo disegna sulla mappa.
Scenari alternativi	1a. L'utente apre il modulo dall'elenco dei centri personalizzati invece che dalla mappa: il sistema propone come posizione il centro della viewport attiva, che l'utente può visualizzare sulla viewport aggiornata prima di salvare. 5a. Nome, indirizzo oppure città sono vuoti: il sistema segnala il campo mancante e resta sul modulo, conservando quanto già inserito. 5b. Né il telefono né l'email sono stati inseriti: il sistema chiede l'inserimento di almeno un contatto. 5c. Il numero di telefono non rispetta il formato nazionale oppure l'indirizzo email non è sintatticamente corretto: il sistema segnala l'errore sul campo interessato. 5d. Nessuna patologia selezionata: il sistema lo segnala e lo richiede. 4a. L'utente annulla: il sistema chiude il modulo senza memorizzare.
Requisiti	RF-25, RF-26, RF-27, RF-28, RF-29

Tabella 3.7: Scheda caso d'uso UC-06.

UC-07 — Ottenere le indicazioni stradali	
Attore primario	Utente
Attori secondari	Applicazioni di navigazione installate sul dispositivo
Precondizioni	La scheda di un centro è aperta e il centro dispone di coordinate valide.
Postcondizioni	Il percorso verso il centro è aperto in un'applicazione di navigazione esterna.
Scenario principale	1. L'utente richiede le indicazioni dalla scheda del centro. 2. Il sistema propone la scelta di un mezzo di trasporto tra automobile, mezzi pubblici, bicicletta e a piedi. 3. L'utente seleziona il mezzo desiderato. 4. Il sistema propone le applicazioni di navigazione. 5. L'utente seleziona un'applicazione. 6. Il sistema crea il collegamento verso quell'applicazione, inserendo le coordinate del centro come destinazione e il mezzo prescelto, e ne delega l'apertura al sistema operativo.
Scenari alternativi	4a. Dispositivo Android: il sistema propone Google Maps e Waze. 4b. Dispositivo iOS: il sistema propone anche Apple Maps. 6a. Viene scelta come applicazione Waze, che accetta nel collegamento la destinazione ma non il mezzo di trasporto: il sistema avvia la navigazione lasciando all'applicazione la sua modalità predefinita. 6b. Viene scelta come applicazione Apple Maps e il mezzo selezionato è la bicicletta, che il collegamento non riconosce: il sistema mostra allora il percorso con un mezzo alternativo, ossia in auto. 6c. Nessuna applicazione è in grado di gestire il collegamento oppure l'apertura fallisce: il sistema avvisa che le indicazioni non possono essere aperte e mantiene la scheda del centro.
Requisiti	RF-32, RF-33

Tabella 3.8: Scheda caso d'uso UC-07.

UC-08 — Contattare il centro	
Attore primario	Utente
Attori secondari	Applicazione telefonica, client di posta elettronica, applicazioni di condivisione
Precondizioni	È aperta la scheda di un centro.
Postcondizioni	L'applicazione assegnata viene aperta con il contatto o con il testo da condividere già predisposto.
Scenario principale	1. L'utente seleziona il marker di un centro. 2. Il sistema apre la scheda con nome, indirizzo e contatti disponibili, mostrando i comandi per chiamare, inviare un'email e condividere. 3. L'utente richiede la chiamata. 4. Il sistema individua il primo numero fra quelli elencati nel campo, ne rimuove i caratteri separatori e apre l'applicazione telefonica con il numero già composto, senza avviare la chiamata.
Scenari alternativi	3a. L'utente richiede l'invio di un'email: il sistema individua il primo indirizzo elencato nel campo e apre il client di posta elettronica con il destinatario precompilato. 3b. L'utente richiede la condivisione: il sistema compone un testo con i dati del centro e un collegamento alla posizione sulla mappa, quindi apre il pannello di condivisione del sistema operativo. 4a. Il campo contiene più recapiti separati da virgole o barre: il sistema ne utilizza il primo. 4b. Dopo la rimozione dei caratteri non ammessi il numero risulta vuoto: il sistema non effettua alcuna azione. 4c. Nessuna applicazione è in grado di gestire il collegamento: l'operazione viene annullata rimanendo sulla scheda corrente.
Requisiti	RF-31, RF-34, RF-35

Tabella 3.9: Scheda caso d'uso UC-08.

Capitolo 4

Progettazione

Questo capitolo raccoglie le decisioni prese prima di scrivere il codice e le ragioni che le hanno motivate. L'ordine segue il percorso naturale del lavoro: prima la scelta della piattaforma e la forma complessiva del sistema, poi l'organizzazione dell'interfaccia, infine la rappresentazione dei dati e la loro conservazione sul dispositivo. Dove una decisione ha comportato una rinuncia, quest'ultima viene dichiarata esplicitamente.

4.1 Architettura del sistema

La premessa che vincola ogni altra scelta è l'assenza di un servizio proprio. Per questo lavoro non è stato realizzato alcun backend: l'applicazione interroga direttamente elenchi pubblicati da terzi, ne normalizza il contenuto e conserva il risultato sul dispositivo. Non esiste quindi un punto in cui i dati vengano ripuliti una volta per tutte a beneficio di tutti gli utenti, e la responsabilità di rendere omogenee sorgenti disomogenee ricade interamente sul client.

La conseguenza è duplice. Da un lato l'App resta autonoma e non richiede infrastruttura da mantenere, il che risulta un vantaggio. Dall'altro l'applicazione è esposta a ogni variazione di formato deciso a monte, e deve continuare a funzionare quando una sorgente diventa irraggiungibile. Entrambe le cose hanno conseguenze dirette sulla progettazione, discusse rispettivamente al paragrafo 4.3.1 e al paragrafo 4.3.2.

L'organizzazione interna segue tre livelli sovrapposti, illustrati nella figura 4.1. La separazione non è imposta da un framework: è una disciplina adottata deliberatamente, perché il modulo risiede in un singolo widget tree e, senza una convenzione esplicita, le responsabilità tenderebbero a mescolarsi.

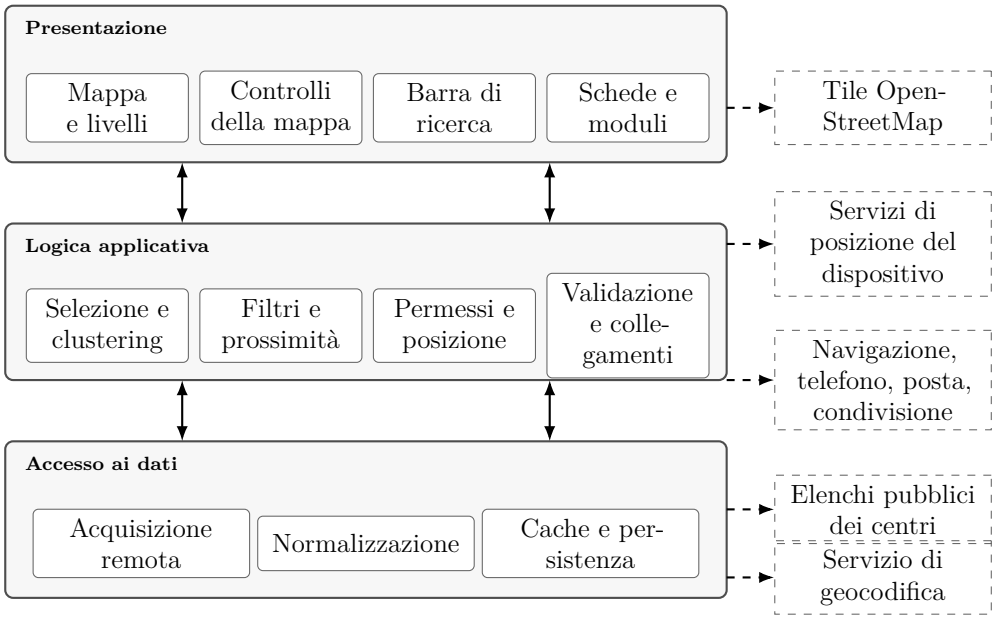


Figura 4.1: Organizzazione del modulo mappa in tre livelli e sistemi esterni interrogati.

Il livello di presentazione contiene la mappa con i suoi livelli sovrapposti, i controlli della mappa, la barra di ricerca e le schede informative. Non prende decisioni: riceve gli elenchi da mostrare e inoltra i gesti dell'utente al livello sottostante.

Il livello di logica applicativa è la parte che decide. Qui vengono stabiliti quali centri ricadono nell'area visibile, come vanno clusterizzati al variare dello zoom, quale struttura risulta la più vicina, se un permesso di posizione va richiesto oppure se occorre rimandare l'utente alle impostazioni di sistema, e se i dati immessi in un modulo sono accettabili.

Il livello di accesso ai dati si occupa dell'acquisizione dalle sorgenti remote, della normalizzazione dei record e della loro conservazione. È l'unico livello che conosce la forma originaria dei dati, e questa è la ragione principale per cui è stato tenuto separato: sopra di esso circolano soltanto record già resi omogenei.

Anche il verso delle frecce della figura dice qualcosa. Gli scambi fra livelli sono bidirezionali perché alle richieste discendenti corrispondono aggiornamenti ascendenti, mentre i collegamenti tratteggiati verso i sistemi esterni sono tutti in uscita: l'applicazione interroga o delega, ma nessun sistema esterno la contatta di propria iniziativa.

4.1.1 Scelta tecnologica: il framework Flutter e Dart

Il requisito di partenza era coprire Android e iOS con un solo sviluppo, avendo a disposizione il tempo di una tesi triennale e un solo sviluppatore. Lo sviluppo nativo separato, con Kotlin da un lato e Swift dall'altro, avrebbe raddoppiato il lavoro su ogni funzione della mappa, dai cluster ai moduli di inserimento, senza portare vantaggi che in questo contesto fossero percepibili.

Fra le soluzioni multipiattaforma la scelta è ricaduta su Flutter, con Dart come linguaggio. Il motivo determinante è il modello di disegno: Flutter non incapsula i componenti nativi del sistema operativo ma disegna la propria interfaccia, e questo rende prevedibile il risultato su entrambe le piattaforme. Per un modulo che sovrappone alla mappa diversi strati di comandi, bolle di clustering e schede a comparsa, la prevedibilità del disegno conta più dell'aderenza a ogni singola convenzione di sistema.

La decisione tecnica più significativa riguarda però la libreria cartografica, e non il framework. La strada consueta sarebbe stata affidarsi al servizio cartografico di Google, che offre cluster e ricerca già pronti. Quella strada richiede tuttavia una chiave di accesso associata a un profilo di fatturazione attivo, con oneri che crescono con le richieste. Per un progetto di tesi, privo di budget e destinato a rimanere una dimostrazione, sarebbe stata una dipendenza sproporzionata e un ostacolo alla riproducibilità del lavoro da parte di chi lo esamina.

La cartografia si appoggia quindi a OpenStreetMap attraverso una libreria che si limita a effettuare il rendering dei tile e a gestire la viewport, senza chiavi né registrazioni. La contropartita è che tutto ciò che il servizio commerciale avrebbe fornito compreso nel prezzo va costruito: il clustering dei marker, la selezione di quelli effettivamente visibili e la ricerca per indirizzo. È per questo motivo che buona parte del capitolo seguente descrive algoritmi che, con una scelta iniziale diversa, non sarebbe stato necessario sviluppare.

Il costo si è manifestato anche in una forma meno prevedibile. Esiste un'estensione della libreria cartografica dedicata proprio al clustering dei marker, che sulla carta avrebbe risolto il problema senza scrivere codice. In pratica quell'estensione richiede una versione della libreria di calcolo geografico incompatibile con quella imposta dalle altre dipendenze del progetto, e il conflitto

non è risolvibile senza retrocedere l'intera catena di dipendenze. Il clustering è stato perciò realizzato a mano, con l'algoritmo descritto al capitolo seguente. È un esempio istruttivo di come, nell'ecosistema dei package, la disponibilità sulla carta di una funzione non coincida con la sua utilizzabilità.

4.1.2 Pattern architetturale e gestione stato

Per la gestione dello stato non è stata adottata alcuna libreria dedicata.

Il modulo mappa ha un solo consumatore dei propri dati e nessuna necessità di condividerli con parti distanti della UI (*User Interface*). Introdurre un contenitore di stato esterno avrebbe aggiunto un livello di indirezione senza eliminare complessità: lo stato sarebbe stato spostato, non semplificato. Gli strumenti offerti dal framework, uno stato locale associato al widget e una notifica puntuale verso i soli elementi interessati, coprono il caso d'uso.

Il punto che ha richiesto attenzione non è infatti dove risieda lo stato, ma quanto spesso venga ridisegnata la mappa. Un aggiornamento indiscriminato a ogni variazione avrebbe ricostruito l'intero albero, tile compresi, a ogni spostamento della viewport. Per questo l'elenco dei centri visibili è stato separato dal resto dello stato e affidato a un oggetto osservabile: quando cambia, si ridisegna il solo livello dei marker, mentre lo sfondo cartografico e i comandi restano intatti.

La tabella 4.1 riassume la distribuzione dello stato e il meccanismo con cui ogni parte propaga i propri cambiamenti.

Elemento di stato	Contenuto	Propagazione
Elenco dei centri visibili	Sottoinsieme dei centri che ricadono nell'area visibile e superano i filtri attivi	Notifica puntuale al solo livello dei marker
Elenco completo dei centri	Tutti i centri della patologia selezionata, già normalizzati	Ridisegno completo, poco frequente
Cache in memoria	Elenchi delle due patologie, indicizzati per nome	Nessuna, è una struttura di servizio
Centri personalizzati	Strutture inserite dall'utente e conservate sul dispositivo	Ridisegno completo, su richiesta esplicita
Posizione dell'utente	Ultima posizione nota, aggiornata dal flusso di localizzazione	Ridisegno completo
Raggio del filtro	Distanza massima ammessa, oppure nessun limite	Ridisegno completo, seguito da nuova selezione
Rotazione della viewport	Orientamento corrente, usato dalla bussola	Ridisegno del solo comando bussola
Suggerimenti di ricerca	Proposte restituite dal servizio di geocodifica	Ridisegno completo, limitato alla barra

Tabella 4.1: Distribuzione dello stato del modulo mappa e modalità di propagazione.

Due scelte accessorie completano il quadro. La prima è la conservazione dello stato quando l'utente lascia la mappa e vi ritorna: senza un accorgimento esplicito il widget verrebbe ricostruito da zero, con una nuova interrogazione della rete e la perdita della viewport. Il modulo dichiara quindi di voler essere mantenuto in vita, e al rientro l'utente ritrova la mappa dove l'aveva lasciata.

La seconda riguarda la frequenza dei ricalcoli. Durante uno spostamento continuo della viewport gli eventi arrivano a ogni fotogramma, e ricalcolare selezione e cluster a quel ritmo sarebbe inutile oltre che oneroso, dato che l'utente vede il risultato solo quando il gesto si ferma. Il ricalcolo è perciò rinviato di una frazione di secondo e rimandato ogni volta che arriva un nuovo evento, così da eseguirlo una sola volta al termine del movimento. Lo stesso accorgimento è applicato, con una pausa più lunga, alle interrogazioni del servizio di geocodifica durante la digitazione, dove la motivazione non è soltanto l'efficienza ma il

rispetto dei limiti d'uso imposti dal servizio.

4.2 Progettazione dell'interfaccia

L'interfaccia è stata pensata attorno a un vincolo semplice: su una mappa il contesto è il contenuto. Se per leggere i dettagli di una struttura l'utente perde di vista la mappa, perde anche la relazione fra quella struttura e le altre, il motivo per cui stava consultando una mappa invece di un elenco. Tutte le decisioni descritte in questa sezione derivano da questa considerazione.

4.2.1 Flusso di navigazione e mappa dell'App

Il modulo occupa una schermata intera, basata su due informazioni: la patologia di interesse e, se già disponibile, la posizione dell'utente. Ricevere la posizione dall'esterno evita di richiedere un permesso che potrebbe già essere stato concesso, e consente di aprire la mappa direttamente sulla zona utile invece che su una viewport predefinita.

Da quel punto in avanti la navigazione si ferma. Nessuna delle funzioni del modulo apre una nuova schermata: consultazione dei dettagli, scelta del percorso, elenco dei centri vicini, inserimento e modifica di una struttura personalizzata avvengono tutte in pannelli che salgono dal basso e coprono solo una parte dello schermo, lasciando visibile la mappa sopra di essi. La figura 4.2 riporta il quadro complessivo.

Questa scelta ha un costo che vale dichiarare. I pannelli modali non si impilano: da un pannello non se ne apre un altro, e dove servirebbe una sequenza di due passi il primo pannello viene chiuso prima di aprire il secondo. Accade nel passaggio dalla scheda di una struttura personalizzata al modulo di modifica, e nella conferma di eliminazione. È un compromesso accettato consapevolmente, perché l'alternativa avrebbe portato a pannelli sovrapposti con una gerarchia di chiusura difficile da prevedere per l'utente.

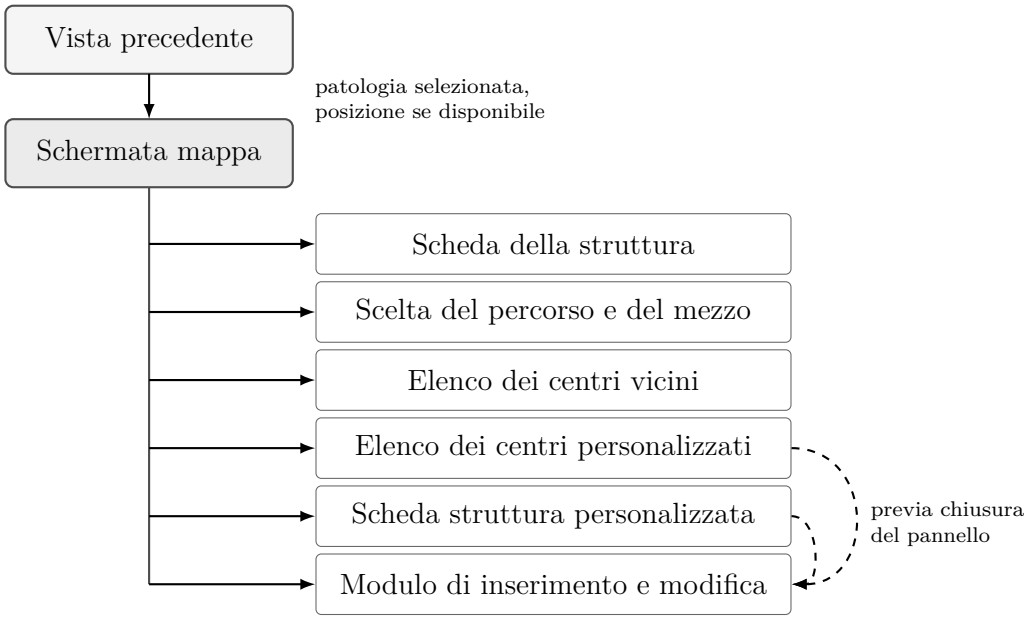


Figura 4.2: Flusso di navigazione del modulo. I passaggi tratteggiati richiedono la chiusura del pannello di partenza.

4.2.2 Progettazione del modulo mappa e filtri

La schermata è costruita per sovrapposizione: la mappa occupa tutto lo spazio disponibile e ogni comando le sta sopra, senza sottrarle superficie. La scelta può sembrare ovvia, ma l'alternativa, cioè riservare ai comandi una fascia propria in alto o in basso, avrebbe ridotto la porzione di territorio visibile proprio sui dispositivi dove è già più scarsa.

La sovrapposizione introduce però un problema che sui banchi di prova non si vede e che compare al primo uso reale: quando l'utente tocca la barra di ricerca, la tastiera di sistema sale dal basso e il sistema di suggerimenti apre un elenco sotto la barra. In quel momento i comandi flottanti si trovano schiacciati fra i due, e alcuni finiscono coperti a metà. La soluzione adottata non è spostare i comandi, che avrebbe prodotto un salto visivo a ogni tocco della barra, ma nasconderli per la durata della ricerca: mentre l'utente sta scrivendo un indirizzo, la sua attenzione è sulla barra e sull'elenco dei risultati, non sui comandi della mappa, che tornano appena la ricerca si conclude.

Alla stessa famiglia di problemi appartiene una seconda decisione: la schermata non si ridimensiona all'apertura della tastiera. Il comportamento predefinito comprimerebbe la mappa nello spazio residuo, cambiando la viewport e costringendo a un nuovo calcolo dei centri visibili proprio mentre l'utente sta digitando.

Disattivando il ridimensionamento la tastiera si limita a coprire la parte bassa della mappa, che resta invariata e ricompare identica alla chiusura.

La tabella 4.2 elenca gli strati sovrapposti nell'ordine in cui vengono disegnati, dal basso verso l'alto. Non tutti sono sempre presenti: alcuni compaiono solo quando la condizione indicata nella terza colonna è soddisfatta, ed è questa presenza condizionata che permette di mantenere leggibile una schermata che, nel caso peggiore, ospita otto elementi sovrapposti.

Strato	**Contenuto**	**Presenza**
Tile cartografici	Immagini del territorio scaricate dal servizio di mappe	Sempre
Cerchio del raggio	Area circolare centrata sull'utente, con bordo e riempimento tenue	Solo con filtro raggio attivo
Centri pubblici	marker dei centri visibili, singoli oppure clusterizzati in bolle numerate	Sempre
Centri personalizzati	marker delle strutture inserite dall'utente	Solo se ne esiste almeno una
Posizione dell'utente	Indicatore circolare della posizione rilevata	Solo con posizione nota
Barra di ricerca	Campo di ricerca per indirizzo, pulsante di ritorno ed elenco dei suggerimenti	Sempre
Colonna dei comandi	Zoom, aggiornamento, bussola, centri vicini, centro più vicino, posizione, centri personali	Nascosta durante la ricerca
Indicatore di caricamento	Avviso testuale con indicatore di attività	Solo durante il recupero dati

Tabella 4.2: Strati sovrapposti della schermata mappa, dal basso verso l'alto.

Il filtraggio per patologia coinvolge due insiemi distinti, gestiti con due meccanismi diversi che l'utente non deve avere motivo di distinguere. I centri pubblici vengono richiesti al servizio remoto già selezionati per patologia, e conservati separatamente per ciascuna: quando l'utente cambia patologia non viene applicato alcun filtro, viene semplicemente sostituito l'insieme di partenza. I centri personalizzati, al contrario, risiedono in un unico archivio locale e portano con sè l'elenco delle patologie a cui si riferiscono, perché una struttura inserita a mano può legittimamente trattarne entrambe; per questi il filtro è valutato al momento del disegno.

L'asimmetria è voluta. Duplicare i centri personalizzati in due archivi separati per allinearli al comportamento dei centri pubblici avrebbe obbligato a inserire due volte la stessa struttura, oppure a mantenere sincronizzate due copie dello stesso dato, con la certezza di vederle divergere. Trattare la patologia come attributo del centro, e non come partizione dell'archivio, evita il problema alla radice; la scelta è discussa insieme al resto del modello nel paragrafo 4.3.1.

Una conseguenza pratica riguarda l'inserimento: un nuovo centro personalizzato nasce associato alla patologia correntemente selezionata, non a entrambe. La preselezione risparmia un tocco nel caso frequente, cioè quando l'utente sta aggiungendo una struttura pertinente a quello che sta consultando, e resta comunque modificabile nel modulo di inserimento. La scelta opposta, associare per difetto tutte le patologie, avrebbe fatto comparire il marker anche in un contesto in cui l'utente non lo aveva richiesto, che è l'errore più fastidioso dei due.

4.3 Modello e persistenza dei dati

Resta da descrivere la parte meno visibile del lavoro, e probabilmente la più delicata: come vengono rappresentati i centri all'interno dell'applicazione, e dove finiscono quando l'applicazione viene chiusa. Il problema nasce dal fatto che le due sorgenti citate al paragrafo 2.2.1 non sono state pensate per essere consumate insieme, e nessuna delle due è stata pensata per questa applicazione.

4.3.1 Rappresentazione e normalizzazione

Le sorgenti differiscono su tre piani, nello specifico differiscono nella forma della risposta, perché una restituisce un oggetto con i record annidati sotto una chiave mentre l'altra restituisce direttamente un elenco. Differiscono nei nomi dei campi, per cui la stessa informazione si chiama in un caso in un modo e nell'altro in un altro. Differiscono, infine, nel grado di strutturazione: una espone i contatti in campi dedicati, l'altra li annega in un frammento di HTML destinato a essere mostrato in una finestra informativa, dal quale vanno estratti.

La tabella 4.3 riporta la corrispondenza fra i campi delle due sorgenti e le informazioni effettivamente impiegate.

Informazione	**Sorgente A**	**Sorgente B**	**Trattamento**
Denominazione	`center_name`	`title`	Primo valore non nullo fra gli alias noti
Indirizzo	`address`	`address`	Composto con città e provincia in un'unica stringa
Città	`city`	assente	Concatenata all'indirizzo quando presente
Provincia	`district`	assente	Aggiunta fra parentesi quando presente
Latitudine	`lat`	`lat`	Conversione tollerante; record scartato se non valida
Longitudine	`lon`	`lng`	Alias distinti, poi conversione tollerante
Telefono	`phone`	dentro `description`	Estratto dal testo con espressione regolare
Posta elettronica	`email`	dentro `description`	Estratto dal testo con espressione regolare
Patologia	implicita nella sorgente	campo `category`	Selezionata a monte, non conservata nel record

Tabella 4.3: Corrispondenza fra i campi delle due sorgenti remote e le informazioni impiegate dall'applicazione.

Davanti a una situazione come questa la soluzione da manuale sarebbe una classe di dominio, con un costruttore per ciascuna sorgente che riceve la mappa grezza e restituisce un oggetto tipizzato. La strada seguita è diversa: i record remoti restano mappe non tipizzate, e la normalizzazione avviene al momento dell'accesso, attraverso catene di alias che interrogano in sequenza i nomi possibili di ciascun campo.

La ragione della scelta è la tolleranza. Una classe di dominio con campi obbligatori impone di decidere in anticipo quali informazioni devono esserci, e fallisce quando la sorgente cambia forma; l'accesso per alias, al contrario, ignora ciò che non riconosce e continua a lavorare su ciò che riconosce. Non avendo il progetto alcun controllo sulle sorgenti, e nessun preavviso sulle loro modifiche, la resistenza al cambiamento è stata preferita alla garanzia sul tipo.

I centri personalizzati hanno una classe propria, con campi tipizzati, serializzazione simmetrica verso e da JSON e un metodo di copia con modifica che conserva l'identificatore.

Campo	Tipo	Comportamento in assenza del dato
Identificatore	testo	Generato dall'istante di creazione espresso in microsecondi
Denominazione	testo	Obbligatorio, validato prima del salvataggio
Indirizzo	testo	Obbligatorio, validato prima del salvataggio
Città	testo opzionale	Le stringhe vuote o di soli spazi diventano assenza di valore
Telefono	testo opzionale	Come sopra; almeno un contatto fra telefono e posta è richiesto
Posta elettronica	testo opzionale	Come sopra
Coordinate	numeri reali	Ricadono a zero in lettura, condizione non producibile dal modulo
Patologie	elenco di testo	In assenza del campo vengono assegnate entrambe le patologie

Tabella 4.4: Struttura di un centro personalizzato e trattamento dei campi mancanti in lettura.

La seconda è l'assegnazione di entrambe le patologie quando il campo manca. Il caso si presenta soltanto leggendo un archivio scritto da una versione precedente, che quel campo non prevedeva. Fra le due possibilità, nascondere quei centri fino a una modifica manuale oppure mostrarli in ogni contesto, è stata scelta la seconda: un centro inserito dall'utente e poi diventato invisibile sarebbe percepito come una perdita di dati, mentre un centro che compare una volta di troppo è al più una imprecisione visibile, e correggibile in due tocchi.

C'è infine un punto in cui i due modelli non si incontrano. Per i centri remoti la patologia non è un attributo del record ma una proprietà dell'insieme da cui il record proviene, poiché la selezione avviene a monte, all'atto della richiesta; per i centri personalizzati è invece un attributo esplicito e multivalore. È la stessa asimmetria osservata al paragrafo 4.2.2, vista dal lato dei dati: due meccanismi diversi che convergono sullo stesso effetto per l'utente, cioè vedere sulla mappa soltanto i centri pertinenti alla patologia scelta.

4.3.2 Strategia di caching e funzionamento offline

Il requisito RNF-10 chiede che l'applicazione resti utilizzabile in assenza di connessione. Non è una richiesta di comodo: la ricerca di un centro avviene tipicamente in viaggio, cioè nella condizione in cui la copertura di rete è meno affidabile, e uno strumento che in quel momento mostra una schermata vuota non è utile.

La conservazione dei dati è organizzata su tre depositi distinti, che rispondono a esigenze diverse e hanno tempi di vita diversi. Il primo è una mappa in memoria, indicizzata per nome della patologia, che evita di ripetere una richiesta già soddisfatta nel corso della stessa sessione. Il secondo è un archivio persistente sul dispositivo, in cui l'ultimo elenco ricevuto viene serializzato in JSON con una chiave derivata dal nome della patologia, e che sopravvive alla chiusura dell'applicazione. Il terzo è l'archivio dei centri personalizzati, che non è una cache ma la sorgente primaria di quei dati, e che quindi non viene mai invalidato.

La distinzione fra il secondo e il terzo deposito è importante e non è soltanto terminologica. Una cache può essere cancellata senza perdita di informazione, perché il dato originale sta altrove; un archivio primario, no. Per questo i due usano chiavi separate e non vengono mai scritti dalla stessa procedura, così che un aggiornamento dell'elenco remoto non possa in nessun caso sovrascrivere ciò che l'utente ha inserito a mano.

Sul deposito persistente è stata adottata una politica che privilegia la reattività: all'avvio l'elenco conservato viene letto e mostrato prima che parta qualunque richiesta di rete, così la mappa si popola immediatamente e la richiesta remota, quando arriva, si limita a sostituire i dati con la versione aggiornata. È la stessa idea del requisito RNF-07, che vieta di far attendere la comparsa della mappa fino al termine della rete, applicata anche ai marker.

Se l'elenco della patologia richiesta è già presente in memoria, la richiesta remota non viene nemmeno tentata: i dati mostrati sono quelli dell'ultimo scaricamento, e restano tali finché l'utente non richiede espressamente un aggiornamento con il comando dedicato.

La figura 4.3 riassume la catena di ricadute che governa l'acquisizione. Ogni anello interviene soltanto quando il precedente non ha prodotto un risultato utilizzabile, e l'ultimo anello, cioè l'avviso all'utente, viene raggiunto solo quando nessun dato è disponibile.

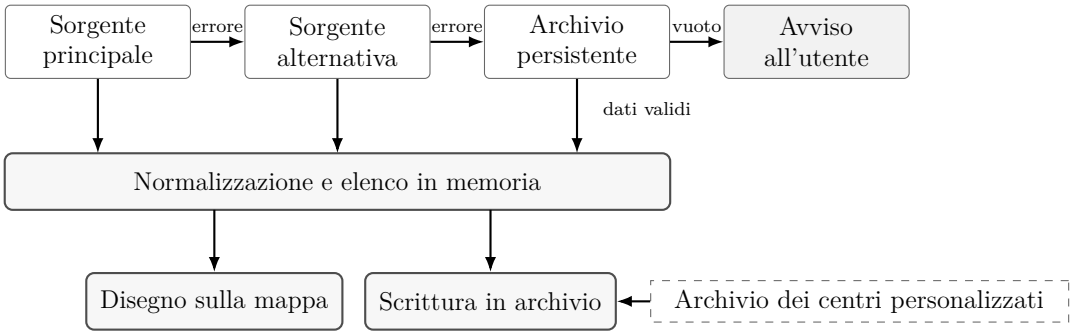


Figura 4.3: Catena dell'acquisizione e utilizzo dei dati. I centri personalizzati ricevono i dati da una fonte propria, indipendente dalla rete.

Nella figura si notano due cose che riassumono l'impostazione dell'intero modulo. La prima è che i centri personalizzati raggiungono il disegno per una via che non attraversa mai la rete: sono l'unica parte del contenuto che resta disponibile e completa anche quando ogni servizio esterno è irraggiungibile. La seconda è che la scrittura in archivio avviene dopo la normalizzazione e non prima, cioè viene conservata la forma originaria del record così come arriva dalla sorgente. La scelta ha una motivazione precisa: conservare la forma normalizzata renderebbe la lettura successiva più semplice; conservare l'originale ha però il vantaggio di non irrigidire l'archivio intorno a una particolare versione della logica di normalizzazione, che può cambiare fra due versioni dell'applicazione mentre i dati sul dispositivo restano quelli.

Un ultimo aspetto riguarda la robustezza in lettura, richiesta dal requisito RNF-11. Un archivio locale può essere corrotto, per un aggiornamento interrotto o per una scrittura non completata, e la lettura di un contenuto non interpretabile solleverebbe un errore proprio durante l'avvio, cioè nel momento peggiore. Tutte le letture dagli archivi locali sono quindi protette e, in caso di contenuto illeggibile, restituiscono un elenco vuoto invece di propagare l'errore: l'applicazione si avvia comunque, eventualmente senza dati, e il primo scaricamento riuscito ricostruisce l'archivio. La verifica di questo comportamento è fra i casi di test descritti al paragrafo 6.2, perché è esattamente il genere di percorso che non viene mai esercitato dall'uso normale e che quindi, senza un test automatico, resterebbe non verificato.

Capitolo 5

Sviluppo

Questo capitolo ripercorre la realizzazione dell'App. L'ordine segue il percorso dei dati, prima l'ambiente e i package, poi l'acquisizione dalle fonti remote e la loro successiva normalizzazione, quindi la memorizzazione in *cache* sul dispositivo, la resa sulla mappa con il clustering dei marker, la localizzazione, la ricerca, i filtri, i centri personalizzati inseriti dall'utente e infine la delega ai servizi del dispositivo. Nei casi in cui il codice realizza dei comportamenti non immediati, ne verranno illustrate prima le ragioni e poi il meccanismo.

5.1 Ambiente di sviluppo e librerie utilizzate

Lo sviluppo si è svolto su Windows 11 con Android Studio, impiegando Flutter e il linguaggio Dart [18, 17]. La verifica durante lo sviluppo è avvenuta prevalentemente su emulatore Android 17 e su dispositivo fisico Android 16, con la limitazione su iOS discussa al paragrafo 6.3. L'interfaccia adotta Material 3 [20], che fornisce già un'implementazione dei componenti impiegati nei pannelli e nei moduli.

Le dipendenze sono state tenute al minimo indispensabile, secondo un criterio di semplicità: ogni package aggiunto è una parte che può causare errori a un aggiornamento e un vincolo sulle versioni degli altri sull'intero progetto. La tabella 5.1 elenca quelle effettivamente impiegate dall'app, con la funzione.

Package	Versione	Funzione nel modulo
`flutter_map`	8.3.1	Rendering dei tile, controllo della viewport, livelli di marker e cerchi del raggio [13]
`latlong2`	0.10.1	Rappresentazione delle coordinate e calcolo delle distanze geodetiche [7]
`geolocator`	14.0.3	Posizione del dispositivo, autorizzazioni di geolocalizzazione, apertura delle impostazioni [9]
`http`	1.6.0	Richieste verso gli elenchi dei centri e verso la geocodifica [10]
`shared_preferences`	2.5.5	Archivio locale per la cache e per i centri personalizzati [15]
`url_launcher`	6.3.2	Apertura del navigatore, percorso, applicazione telefonica e client di posta [16]
`share_plus`	13.3.0	Condivisione della scheda di un centro [14]

Tabella 5.1: Dipendenze impiegate dal modulo mappa e relativa funzione.

Tutte le dipendenze sono dichiarate con un vincolo che ammette futuri aggiornamenti compatibili, tranne quella per la condivisione, fissata a una versione esatta.

Sul fronte del supporto multipiattaforma, sia per Android sia per iOS, l'app richiede alcune dichiarazioni esplicite dei permessi, riassunte nella tabella 5.2. Su Android sono necessari l'accesso alla rete e l'accesso alla posizione; a questi si aggiunge la dichiarazione degli schemi di collegamento che l'applicazione intende interrogare, introdotta nelle ultime versioni del sistema, senza la quale la verifica della disponibilità di un'applicazione esterna (per esempio, Google Maps) fallirebbe anche quando l'applicazione è installata. Su iOS occorre invece motivare testualmente l'uso della posizione, mostrata all'utente nel momento in cui viene richiesta [21, 8].

Piattaforma	Dichiarazione	Motivo
Android	`INTERNET`	Richieste verso elenchi, *endpoint* di tipo *GET*, geocodifica e tile
Android	`ACCESS_FINE_LOCATION`	Posizione precisa
Android	`ACCESS_COARSE_LOCATION`	Alternativa quando l'utente concede l'accesso solamente alla posizione approssimativa
Android	schemi `https`, `tel`, `mailto`	Verifica della disponibilità delle applicazioni esterne da interrogare
iOS	`NSLocationWhenInUse-UsageDescription`	Testo mostrato all'utente alla richiesta del permesso per l'accesso alla posizione

Tabella 5.2: Permessi richiesti dalle due piattaforme e relativa motivazione.

Il modulo è realizzato come un albero di widget, che mantiene l'elenco dei centri, la posizione dell'utente, i filtri attivi e i riferimenti ai controller della mappa e del campo per la ricerca. Alla classe dello stato sono applicate due estensioni: la prima consente di dichiarare che il widget deve essere mantenuto in vita quando l'utente perde il *focus* dalla schermata, la seconda fornisce il supporto temporale necessario alle animazioni, impiegato dal comando della bussola descritto al paragrafo 5.3.1.

5.2 Acquisizione e normalizzazione dei dati

Ogni accorgimento descritto in questo paragrafo corrisponde a un caso reale osservato durante lo sviluppo.

5.2.1 Interrogazione delle fonti e fallback

Le due patologie sono servite da due fonti indipendenti. Quella per la dermatite atopica espone un servizio che restituisce un oggetto con *record* annidati raggruppati da una chiave, e non richiede alcun criterio particolare. Quella per la psoriasi è esposta da un sistema CMS (*Content Management System*), che restituisce l'insieme completo dei marker per una mappa, dai quali occorre selezionare la categoria di interesse: la stessa mappa ospita sia i centri per la psoriasi sia quelli per la dermatite atopica, distinti da un campo di categoria che può contenerne più di uno, separati da virgola.

Due particolarità di questa seconda fonte hanno richiesto delle correzioni. La prima è che il servizio nega la risposta alle richieste che non dichiarano un'intestazione di identificazione compatibile con quella di un normale browser; la richiesta la dichiara quindi esplicitamente. La seconda è che il corpo (*body*) della risposta arriva preceduto da un BOM (*Byte Order Mark*), che il decodificatore non rimuove e che, rimanendo come primo carattere della stringa, rende l'intero documento non interpretabile. La decodifica avviene perciò in due passi: interpretazione dei byte come testo, tollerando le sequenze invalide invece di interrompersi, e rimozione del BOM iniziale quando presente.

Quando la fonte dei centri per la psoriasi non risponde, o risponde con un contenuto che non produce alcun record utilizzabile, l'applicazione non si ferma ma passa in *fallback* (*asset* di riserva) sull'altra fonte. Questo crea un compromesso, in parte trascurabile, in quanto una parte considerevole delle strutture della prima fonte segue entrambe le patologie.

Ogni richiesta ha un *timeout* di quarantacinque secondi, come previsto dal requisito RNF-08.

Un ultimo accorgimento riguarda il precaricamento. Quando l'elenco della patologia richiesta è stato ottenuto, l'applicazione avvia in *background* il recupero dell'altra, senza attenderne il risultato e senza segnalarne l'operazione. Il passaggio da una patologia all'altra avviene infatti con un comando a due sole posizioni, e senza precaricamento comporterebbe un'attesa della risposta dalla rete a ogni cambio; con il precaricamento il passaggio è istantaneo dopo la prima volta. L'eventuale errore del precaricamento viene ignorato, perché riguarda dati che l'utente non ha ancora richiesto di vedere.

La figura 5.1 riassume la sequenza di interrogazione appena descritta, e mostra i tre punti in cui il flusso di elaborazione si dirama: il successo della richiesta, l'attivazione del *fallback* e l'avvio del precaricamento dell'altra patologia.

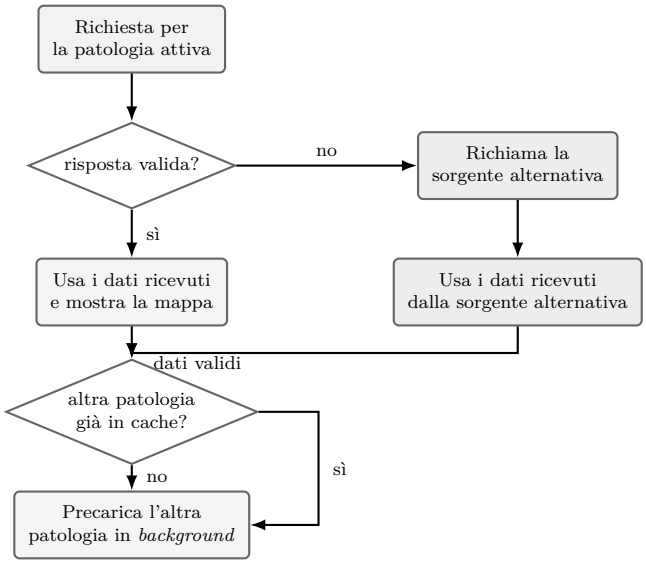


Figura 5.1: Sequenza di interrogazione delle sorgenti remote. La diramazione principale avviene sull'esito della prima richiesta; il *fallback* converge sul nodo che governa il precaricamento.

5.2.2 Normalizzazione dei record

La normalizzazione ha tre compiti distinti:

1. Ricondurre a un nome unico i campi che le due fonti ritornano in formati diversi

2. Recuperare le informazioni, specialmente quando una delle due non le ritorna in forma strutturata

3. Riconoscere i record inutilizzabili scartandoli senza interrompere l'elaborazione degli altri, come da requisito RNF-09.

La figura 5.2 mostra la pipeline che ne risulta: ogni record attraversa in sequenza i tre compiti, e lo scarto di un record non valido interessa soltanto il ramo che lo contiene, senza fermare l'elaborazione degli altri.

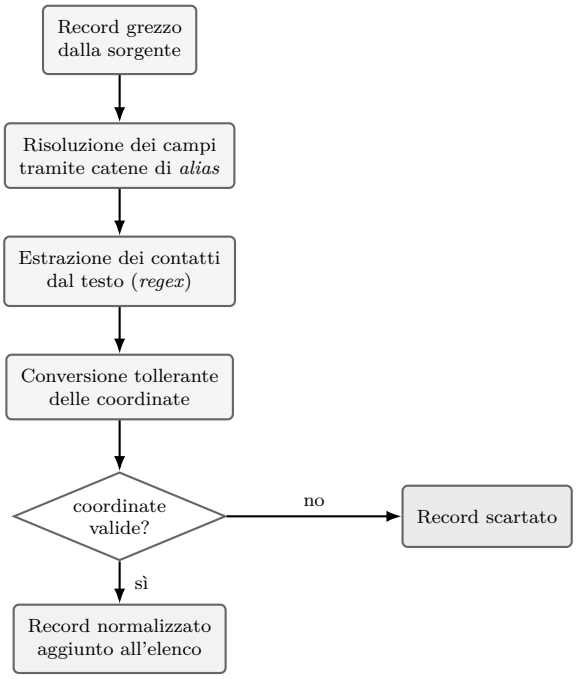


Figura 5.2: Pipeline di normalizzazione di un record: i tre compiti in sequenza e lo scarto per coordinate invalide, che non interrompe l'elaborazione degli altri record (requisito RNF-09).

Il primo punto si risolve con le catene di *alias* già discusse al paragrafo 4.3.1, ove la denominazione viene quindi cercata in sequenza fra i nomi già noti, la longitudine viene cercata sotto due chiavi distinte, e in entrambi i casi il primo valore trovato vince sul secondo.

Il secondo punto riguarda i contatti, come email e numeri di telefono, dalla fonte dei centri per la psoriasi, che non li espone in campi propri ma li lascia dentro un frammento di codice HTML. Il recupero avviene in due passaggi: rimozione dei *marker* dal testo e relativa ricerca dei recapiti tramite *regex* (espressioni regolari).

Il terzo punto riguarda le coordinate. Alcuni record riportano valori con più di un solo separatore decimale, come conseguenza di un errore di trascrizione della fonte. La normale conversione su un valore di questa tipo fallisce restituendone uno nullo, e il record verrebbe scartato pur contenendo un'informazione che sarebbe in realtà recuperabile: le cifre ci sono tutte, ma è il formato a essere errato. La conversione, riportata in 1, conserva il primo separatore e rimuove i successivi, riportando la stringa a una forma corretta e interpretabile.

```dart
double? _parseCoordinate(dynamic value) {
  if (value == null) return null;
  var s = value.toString().trim();
  if (s.isEmpty) return null;
  final parts = s.split('.');
  if (parts.length > 2) {
    s = '${parts.first}.${parts.skip(1).join()}';
  }
  return double.tryParse(s);
}
```

Listato 1: Conversione di una coordinata, con recupero dei valori che riportano più separatori decimali.

La funzione restituisce un valore nullo quando la conversione non è possibile. Non è previsto un valore predefinito. Questo per un buon motivo: un valore predefinito, per esempio l'origine delle coordinate, collocherebbe il centro in una posizione totalmente errata, dove comparirebbe come un marker perfettamente legittimo ma nella realtà, inutile. Restituire un valore nullo invece obbliga ogni chiamante, nel nostro caso, a ignorare il record. È l'applicazione pratica del requisito RF-04.

5.2.3 Salvataggio sul dispositivo

Il salvataggio locale impiega l'archivio di coppie chiave-valore fornito dalla piattaforma, che memorizza tipi elementari e che viene quindi usato per custodire i dati serializzati in formato JSON. Per i tre archivi *cache* descritti al paragrafo 4.3.2 le chiavi sono distinte, come riassunto nella tabella 5.3.

Archivio	**Chiave**	**Contenuto**
Cache dell'elenco, una per patologia	Prefisso fisso e nome della patologia	Ultimo elenco remoto scaricato, in forma originale
Centri personalizzati	Chiave fissa con suffisso di versione (*version tag*)	Voci inserite dall'utente, in forma tipizzata

Tabella 5.3: Chiavi degli archivi locali: due per gli elenchi delle patologie, una per i centri personalizzati.

Il *version tag* nella chiave dei centri personalizzati è necessario per il seguente motivo. L'archivio contiene anche dati che l'utente ha inserito a mano e che non possono essere riscaricati; se una futura versione dovesse cambiare la

struttura del record in modo da renderlo incompatibile, quel suffisso permette di leggere il vecchio contenuto sotto la precedente chiave e di convertirlo, invece di trovarsi davanti dati non interpretabili sotto la nuova chiave attesa. È lo stesso ragionamento che si applica alla migrazione di uno schema di database, ridotto alla scala dell'app.

All'avvio vengono eseguite le operazioni definite al paragrafo 4.3.2 per la gestione della *cache*. Vengono prima letti i *record* salvati sul dispositivo per entrambe le patologie e riportate nella mappa, poi viene effettuata una richiesta alla fonte. Il vantaggio è che all'apertura la mappa ha già dei marker da disegnare, e l'attesa della risposta dalla rete non blocca l'interazione; il precaricamento dell'altra patologia, descritto al paragrafo 5.2.1, viene eseguito sempre basandosi su questo meccanismo.

Il salvataggio in locale viene avviato subito prima di aggiornare lo stato da cui dipende il ridisegno della mappa. In questo modo l'utente vede subito i marker senza dover aspettare che vengano prima scritti su disco, e un eventuale errore di scrittura non ha conseguenze sulla sessione in corso, poiché i dati sono comunque presenti in memoria. L'errore viene comunque registrato nei *log*, così che uno sviluppatore possa diagnosticare e trovare una soluzione al problema.

Tutte le letture in locale sono racchiuse in una gestione degli errori che, in caso di contenuto illeggibile o d'errore, restituisce un elenco vuoto. La lettura inoltre verifica che il contenuto interpretato sia effettivamente un elenco, perché un documento JSON sintatticamente valido ma di forma diversa o malformato, per esempio un oggetto, supererebbe l'interpretazione per poi fallire durante un utilizzo reale. I due casi sono verificati fra i test descritti al paragrafo 6.2.

5.3 Caricamento mappa e gestione dei marker

5.3.1 Livelli, marker e controlli della mappa

La schermata è costruita per sovrapposizione, secondo l'ordine di disegno riportato nella tabella 4.2. Alla base sta il livello dei tile, che richiede due soli parametri: l'URL del template da cui ricavare l'immagine di ciascun tile a

partire da un determinato zoom, riga e colonna, secondo lo schema descritto al paragrafo 2.3, e un identificativo dell'applicazione.

Sopra i tile, quando il filtro per raggio è attivo, viene disegnato il cerchio che ne rappresenta l'estensione. Il raggio è espresso in chilometri e non in punti dello schermo, così che il cerchio resti ancorato al territorio e si dilati o si contragga con lo zoom, come l'utente si aspetterebbe da una grandezza geografica.

Il livello dei marker dei centri pubblici è l'unico che si appoggia a un costruttore che si lega all'oggetto osservabile discusso al paragrafo 4.1.2. A livello pratico l'unica conseguenza è che l'aggiornamento dell'elenco dei centri visibili ridisegna quel solo livello: i tile non vengono nuovamente richiesti, i comandi non vengono ricostruiti, e il costo del ridisegno resta proporzionale al numero di marker e non alla complessità dell'intera schermata.

La selezione dei centri da disegnare avviene confrontando le coordinate di ciascun record con i limiti della viewport attiva, allargati di metà della loro estensione su ciascun lato. se non ci fosse un tale margine, un marker appena fuori dal bordo comparirebbe soltanto al termine dello spostamento, producendo un effetto di comparsa improvvisa lungo i lati dello schermo. Con un margine pari a metà dell'ampiezza per lato l'area interrogata ha una superficie più ampia: si disegnano più marker di quelli necessari, in cambio di un ingresso graduale ai bordi. Inoltre, quando il filtro per raggio è attivo, viene scartato ciò che si trova al di fuori della distanza ammessa, così che le due condizioni siano valutate in un'unica elaborazione senza dover rileggere più volte l'intera lista dei centri.

L'aggiornamento è governato da due meccanismi che agiscono in sequenza. Il primo è il rinvio temporale: gli eventi della mappa sono numerosi durante lo spostamento su di essa, quindi invece di aggiornarla a ogni movimento viene avviato un timer e alla sua scadenza la mappa viene aggiornata. Il secondo è il confronto con il risultato precedente: se l'elenco aggiornato coincide con quello già in uso, ne viene annullato l'aggiornamento.

Non tutti gli eventi della mappa provocano un ricalcolo. Vengono considerati solo quelli che segnalano la conclusione di un movimento, uno spostamento, uno zoom per doppio tocco, l'utilizzo di un controllo; gli eventi intermedi, emessi a ogni fotogramma mentre il dito è ancora sullo schermo, vengono ignorati. È la

realizzazione del requisito RNF-06. La figura 5.3 riassume l'intera catena, dal tipo di evento al ridisegno del solo livello dei marker.

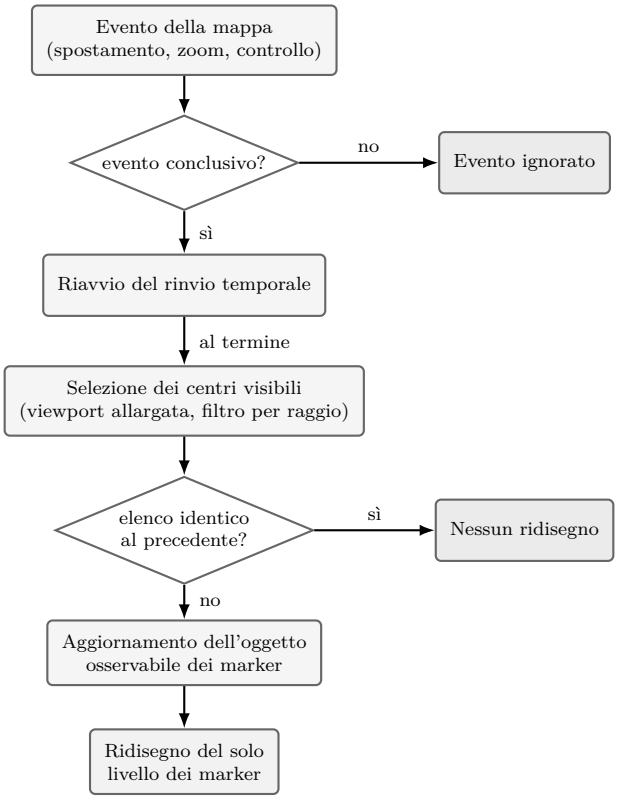


Figura 5.3: Catena di aggiornamento dei marker: solo gli eventi conclusivi riavviano il rinvio temporale, e il ridisegno avviene soltanto se l'elenco selezionato è cambiato.

Un controllo particolare della mappa è la bussola, in particolare per quanto riguarda il riallineamento al valore predefinito dopo una rotazione. Il comando compare soltanto quando si ruota la mappa di oltre due gradi, per questione pratica ed estetica, e la rotazione viene registrata nello stato solo quando si discosta di oltre mezzo grado dall'ultimo valore noto, per non provocare un aggiornamento sulla schermata a ogni fotogramma della rotazione.

Per far ruotare la mappa a nord, non viene spostata istantaneamente a zero, ma viene interpolata seguendo il percorso più breve. Se la mappa è ruotata di, per esempio, trecentocinquanta gradi, l'orientamento corretto si raggiunge ruotando di dieci gradi in avanti e non di trecentocinquanta all'indietro, e senza un controllo esplicito a livello di codice l'animazione compierebbe il giro lungo. Viene quindi calcolato prima di essere animato lo scarto basandosi sull'intervallo fra meno e più centottanta gradi, con l'espressione riportata in 2.

```dart
final double start = _mapController.camera.rotation;
double delta = -start;
delta = ((delta + 180) % 360) - 180;
final double target = start + delta;
```

Listato 2: Calcolo percorso più breve della bussola per puntare a nord.

L'animazione dura trecentocinquanta millisecondi con una funzione di *easing* (interpolazione), al termine della quale la rotazione viene impostata esattamente a zero.

5.3.2 Algoritmo di clustering dei marker

La funzionalità di clustering è stata riscritta in modo nativo, come discusso al paragrafo 4.1.1, a causa di problemi di compatibilità tra le versioni. Questo serve quando i centri dermatologici si concentrano, per esempio, nelle aree urbane, come nella zona di Milano o di Roma dove decine di marker finiscono a pochi punti di distanza l'uno dall'altro, sovrapponendosi fino a rendere impossibile sia contarli e sia selezionarli.

L'approccio adottato è il *grid-based clustering* (clustering basato a griglia). Lo spazio delle coordinate viene suddiviso in celle quadrate, ogni centro visibile viene assegnato alla cella che lo contiene, e ogni cella non vuota produce un cluster collocato nel baricentro dei propri membri. Le celle con un solo membro danno luogo a un normale marker, quelle con più membri a un cluster (una bolla) che ne riporta il numero.

La dimensione della cella dipende dallo zoom, perché la sovrapposizione è un fenomeno che avviene sullo schermo e non sul territorio: due centri distanti dieci chilometri si sovrappongono in una vista ampia ma sono ben separati in una vista ravvicinata. La formula impiegata è

$$c(z) = \frac{84}{2^z} \tag{5.1}$$

dove z è il livello di zoom corrente e $c(z)$ il lato della cella in gradi. La struttura della formula segue direttamente la relazione esponenziale dello schema slippy map richiamata al paragrafo 2.3: se ogni incremento unitario dello zoom

raddoppia il numero di tile per lato, il lato della cella deve dimezzarsi allo stesso modo perché il suo effetto visivo resti invariato.

Gli errori dipendenti dallo zoom si annullano, e il lato della cella sullo schermo risulta costante a qualunque livello di ingrandimento. È questa la proprietà per cui si rende necessaria la formula; la cella non è una distanza geografica ma una distanza sullo schermo, di circa sessanta punti, cioè circa un quarto del lato di un tile. La costante 84 non è quindi un valore calcolato privo di senso, ma il numero di gradi che a ogni zoom corrisponde a quella distanza sullo schermo.

L'asse verticale si comporta in modo leggermente diverso, e la differenza è una conseguenza della proiezione. Le celle sono dunque quadrate in gradi ma leggermente più alte che larghe sullo schermo, con una distorsione che cresce verso nord. La tabella 5.4 riporta i valori a diversi livelli di zoom.

Zoom	**Lato (gradi)**	**Lato N-S**	**Lato E-O**	**Largh. (pt)**	**Alt. (pt)**
6	1,312500	146,1 km	103,3 km	59,7	84,5
9	0,164063	18,3 km	12,9 km	59,7	84,5
12	0,020508	2283 m	1614 m	59,7	84,5
14	0,005127	571 m	404 m	59,7	84,5
16	0,001282	143 m	101 m	59,7	84,5
18	0,000320	36 m	25 m	59,7	84,5

Tabella 5.4: Dimensione della cella di clustering ai vari livelli di zoom. Le distanze in metri e l'altezza in punti sono calcolate alla latitudine di 45 gradi.

Il codice 3 riporta la costruzione dei cluster. L'assegnazione alla cella avviene dividendo le coordinate per il lato e troncando all'intero inferiore; le celle sono conservate in una struttura associativa a due livelli, indicizzata prima per colonna e poi per riga, e ogni cella non vuota produce un cluster nel baricentro dei propri membri.

```
List<_Cluster> _buildClusters(List<dynamic> centers, double zoom) {
  final double cell = 84.0 / math.pow(2, zoom);
  final Map<int, Map<int, List<dynamic>>> grid = {};
  for (final d in centers) {
    final lat = _parseCoordinate(d['lat']);
    final lon = _parseCoordinate(d['lon'] ?? d['lng']);
    if (lat == null || lon == null) continue;
    final gx = (lon / cell).floor();
    final gy = (lat / cell).floor();
    (grid[gx] ??= {}).putIfAbsent(gy, () => []).add(d);
  }
  final clusters = <_Cluster>[];
  grid.forEach((_, col) {
    col.forEach((_, members) {
      double sumLat = 0, sumLon = 0;
      for (final d in members) {
        sumLat += _parseCoordinate(d['lat'])!;
        sumLon += _parseCoordinate(d['lon'] ?? d['lng'])!;
      }
      clusters.add(_Cluster(
        LatLng(sumLat / members.length, sumLon / members.length),
        members,
      ));
    });
  });
  return clusters;
}
```

Listato 3: Costruzione dei cluster a griglia dipendente dallo zoom.

Il costo dell'algoritmo è lineare nel numero di centri esaminati, perché ciascuno viene collocato nella propria cella con un numero costante di operazioni e nessun punto viene confrontato con nessun altro.

La figura 5.4 riassume il principio con un breve esempio, dove le coordinate sono state riportate in unità di cella per rendere la lettura indipendente dal livello di zoom.

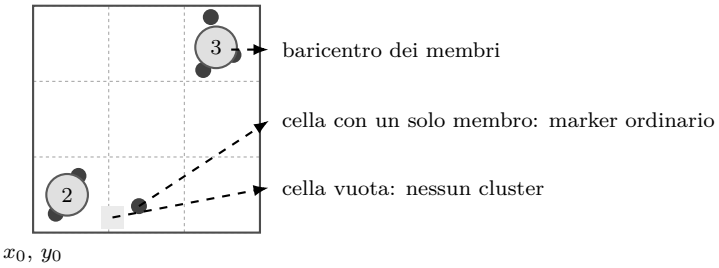


Figura 5.4: Principio del clustering a griglia: le celle sono ancorate all'origine delle coordinate e ogni cella non vuota produce un cluster nel baricentro dei propri membri, oppure un marker ordinario qualora contenga un solo centro.

La selezione della bolla, per esempio tramite tocco, provoca un avvicinamento di due livelli di zoom; il limite massimo è il diciottesimo livello. Il valore rispetta il requisito RF-10, che chiede la scomposizione del clustering negli elementi che lo costituiscono. Due livelli comportano, per l'equazione 5.1, una riduzione del lato della cella a un quarto e quindi la suddivisione della cella di partenza in un massimo di sedici nuove celle. Un singolo livello di zoom sarebbe stato insufficiente in molte configurazioni urbane, imponendo all'utente più tocchi successivi sulla stessa bolla; tre avrebbero fatto perdere il contesto circostante.

5.4 Localizzazione e gestione dei permessi

La posizione dell'utente serve a tre cose: centrare la mappa all'apertura, calcolare le distanze e valutare il filtro per raggio. Nessuna delle tre è indispensabile al funzionamento del modulo, e questa è la premessa che ha guidato l'intera gestione dei permessi: l'applicazione deve restare utilizzabile anche quando la posizione non c'è, come prevede il requisito RF-14.

Il permesso non viene richiesto all'avvio. La schermata riceve dalla vista precedente la posizione, se questa è già disponibile, e in quel caso si limita ad avviare il flusso di aggiornamento senza alcuna richiesta all'utente; quando invece la posizione non arriva, la richiesta viene rinviata al momento in cui l'utente attiva una funzione che ne ha bisogno, cioè il ritorno alla propria posizione, il centro più vicino o l'elenco dei centri vicini. Traduce il requisito RF-12, ed è anche una scelta di merito: un permesso chiesto prima che ne sia evidente il motivo viene negato con maggiore frequenza di uno chiesto in risposta a un comando esplicito.

La procedura di acquisizione attraversa gli stati riportati nella figura 5.5. I casi da distinguere sono quattro e non uno, e ciascuno richiede una risposta diversa: il servizio di localizzazione del dispositivo può essere disattivato del tutto, il permesso può non essere ancora stato chiesto, può essere stato negato una volta, oppure può essere stato negato in modo permanente.

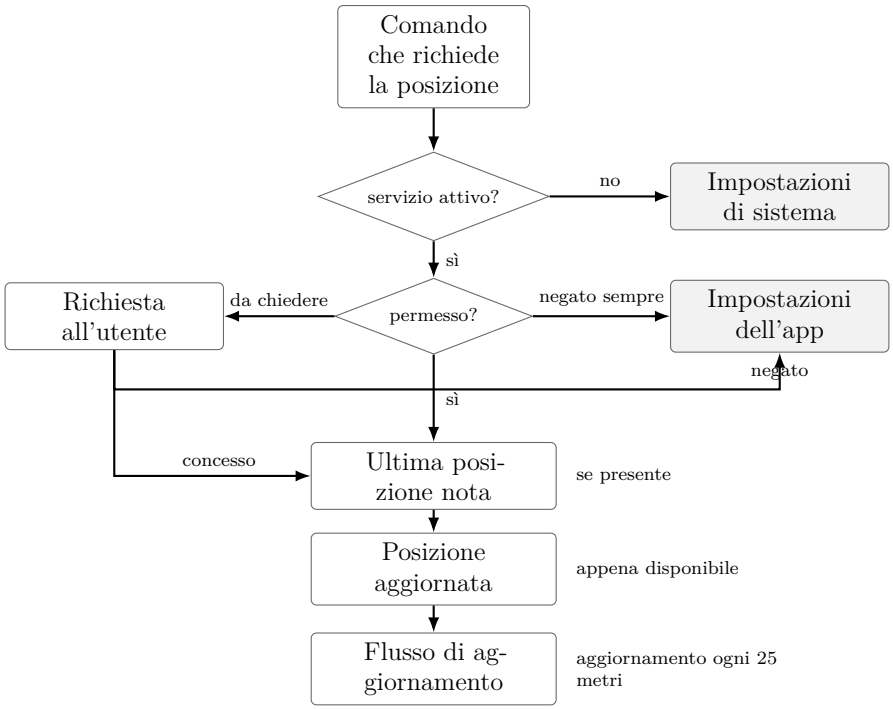


Figura 5.5: Stati attraversati nell'acquisizione della posizione, sia in caso di posizione concessa che negata.

La distinzione fra il servizio disattivato e il permesso negato in modo permanente non è una sottigliezza, perché le due condizioni si risolvono in schermate di sistema diverse: la prima nelle impostazioni di localizzazione del dispositivo, la seconda nella scheda dell'applicazione. Portare l'utente nella schermata sbagliata equivale a non aiutarlo: l'interruttore che cerca non si trova lì. Il negato permanente, in particolare, si riconosce dal fatto che da quello stato il sistema operativo non ripresenta più la richiesta: insistere con una nuova domanda non produrrebbe alcuna finestra e l'utente vedrebbe un comando che non fa nulla. In quel caso l'unica strada è il collegamento diretto alle impostazioni, come richiesto dal requisito RF-15.

L'acquisizione riuscita procede in due tempi, e la ragione è la reattività percepita. La posizione di alta precisione richiede alcuni secondi, durante i quali l'interfaccia non avrebbe nulla da mostrare; l'ultima posizione nota, conservata

dal sistema operativo, è invece disponibile immediatamente ed è sufficiente per centrare la mappa in modo approssimativo. Viene quindi impiegata subito, se presente, e sostituita dalla posizione precisa quando questa arriva. Al termine viene attivato un flusso di aggiornamento con una soglia di venticinque metri, valore scelto per aggiornare l'indicatore durante uno spostamento reale senza reagire alle oscillazioni del rilevamento a utente fermo.

Un accorgimento minore ma necessario riguarda le richieste sovrapposte. I comandi che necessitano della posizione sono tre e sono tutti raggiungibili in rapida successione; senza una protezione, due pressioni ravvicinate avvierebbero due procedure concorrenti, con la possibilità di presentare all'utente due finestre di richiesta sovrapposte. Un indicatore di procedura in corso impedisce l'avvio di una seconda acquisizione mentre la prima non si è conclusa.

5.5 Ricerca per indirizzo e geocodifica

La ricerca per indirizzo si appoggia al servizio di geocodifica dell'ecosistema OpenStreetMap descritto al paragrafo 2.3.1, interrogato in due modalità distinte. Durante la digitazione la richiesta chiede fino a cinque risultati con i dettagli dell'indirizzo, che alimentano l'elenco dei suggerimenti; alla conferma esplicita, con il tasto di invio o con il comando di ricerca, la richiesta chiede il solo primo risultato e sposta direttamente la viewport.

Il vincolo che governa questa parte non è tecnico ma di conformità. La politica d'uso del servizio ammette al massimo una richiesta al secondo, richiede che l'applicazione si identifichi e invita a conservare i risultati per non ripetere interrogazioni identiche [26]. Una barra di ricerca che interrogasse il servizio a ogni carattere digitato supererebbe quel limite dopo poche lettere, e il requisito RF-17 nasce proprio da questa considerazione.

L'interrogazione è quindi rinviata di mezzo secondo dall'ultimo carattere digitato, e ogni nuovo carattere annulla il rinvio pendente e ne programma un altro. La conseguenza è che una digitazione fluida non produce alcuna richiesta fino alla prima pausa, e che il numero di richieste dipende dal numero di pause invece che dal numero di caratteri. Il valore di mezzo secondo è il doppio di quello impiegato per il ricalcolo dei marker descritto al paragrafo 5.3.1, e la differenza è voluta: nel primo caso si tratta di rispettare un limite d'uso esterno,

nel secondo soltanto di non sprecare calcolo locale. La figura 5.6 mostra il comportamento della digitazione con il rinvio: ogni nuovo carattere annulla il rinvio pendente e ne programma uno nuovo, quindi la richiesta parte solo alla prima pausa.

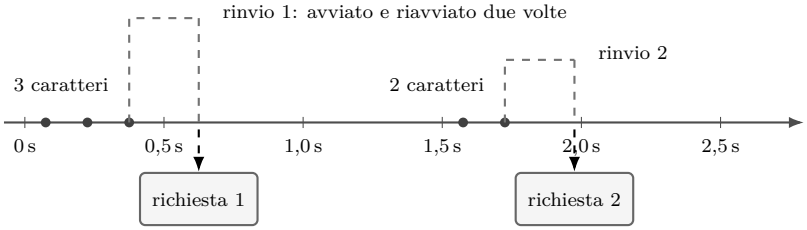


Figura 5.6: Comportamento del rinvio sull'input di ricerca: la richiesta parte soltanto quando la digitazione si interrompe per mezzo secondo, quindi il numero di richieste dipende dalle pause e non dai caratteri.

Sul piano dell'interfaccia la ricerca ha imposto la soluzione del problema di sovrapposizione descritto al paragrafo 4.2.2. Mentre i comandi sono nascosti vengono anche resi insensibili al tocco: un elemento reso trasparente resta altrimenti capace di intercettare i tocchi, e l'utente si troverebbe a premere un pulsante invisibile mentre cerca di selezionare un suggerimento.

5.6 Filtri e ricerca per prossimità

5.6.1 Distanze geodetiche e filtro per raggio

Il requisito RF-22 chiede che il calcolo delle distanze tenga conto della curvatura terrestre, e la richiesta non è un eccesso di rigore. Trattare latitudine e longitudine come coordinate cartesiane, cioè applicare il teorema di Pitagora ai gradi, produce un errore che dipende dalla latitudine e che alle latitudini italiane è dell'ordine del trenta per cento sulle distanze in direzione est-ovest, perché un grado di longitudine misura circa settantotto chilometri a Milano e centoundici all'equatore. Su un elenco ordinato per distanza un errore di quell'entità non si limita a sbagliare i numeri: cambia l'ordine dei risultati, ed è quindi visibile all'utente come un centro più lontano collocato prima di uno più vicino.

Il calcolo impiega la funzione di distanza della libreria geografica, che applica la formula dell'emisenoverso [3]. La formula fornisce la lunghezza dell'arco di cerchio massimo fra due punti su una sfera, e per le distanze in gioco in questa

applicazione, dell'ordine delle decine o centinaia di chilometri, l'approssimazione sferica è più che adeguata: lo scostamento rispetto a un calcolo su ellissoide resta entro qualche parte per mille, ben al di sotto della precisione con cui i risultati vengono presentati all'utente.

La presentazione delle distanze segue tre scaglioni, scelti in base a quello che l'utente può fare con il dato. Sotto il chilometro la distanza è espressa in metri arrotondati, perché a quella scala la differenza fra trecento e ottocento metri decide se andare a piedi. Fra uno e dieci chilometri viene usato un decimale. Oltre i dieci chilometri l'arrotondamento è al chilometro intero, perché i decimali su una distanza di cinquanta chilometri suggerirebbero una precisione che il dato non ha, trattandosi comunque di una distanza in linea d'aria e non di un percorso stradale.

La figura 5.7 rappresenta i tre scaglioni come fasce adiacenti su un asse, associate ciascuna all'unità di misura e all'azione tipica che il dato suggerisce all'utente.

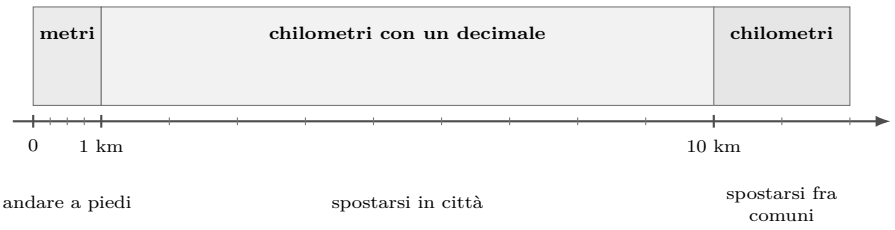


Figura 5.7: Scaglioni della distanza, con l'unità di misura e l'azione tipica associata a ciascuno.

Tutte le distanze mostrate sono geodetiche, cioè in linea d'aria, mentre l'utente è interessato al percorso reale, che in territorio montano o attraverso un centro urbano può essere notevolmente più lungo. Un calcolo su rete stradale richiederebbe un servizio di instradamento, cioè un'altra dipendenza esterna con i propri limiti d'uso, e non è stato introdotto; l'ordinamento per distanza in linea d'aria resta comunque una approssimazione utile del più vicino, e la distanza reale viene poi mostrata dall'applicazione di navigazione a cui il percorso viene delegato.

Il filtro per raggio ammette quattro valori, cioè nessun limite, venticinque, cinquanta e cento chilometri, presentati come scelte a chip invece che con un selettore continuo. La decisione risponde al contesto d'uso: un selettore a scorrimento richiede una regolazione fine con il pollice e restituisce valori

arbitrari come trentasette chilometri, che non corrispondono a nessuna domanda che l'utente si ponga davvero. Valori discreti si selezionano con un tocco solo e coprono le tre domande effettive, cioè la città, la provincia e la regione.

L'attivazione del filtro produce tre effetti coordinati: la selezione dei centri visibili scarta quelli oltre la distanza, l'elenco dei centri vicini si restringe di conseguenza, e sulla mappa compare il cerchio che ne rappresenta l'estensione. Il terzo effetto è quello che rende comprensibili i primi due, e non a caso il requisito RF-24 lo prevede: senza una rappresentazione grafica del raggio l'utente non ha modo di collegare la scomparsa di alcuni marker al filtro che ha appena impostato, e la mappa sembra semplicemente aver perso dei dati.

Il filtro si applica anche ai centri inseriti dall'utente, come impone il requisito RF-23, ma per una via diversa. I centri pubblici lo attraversano nella selezione per viewport descritta al paragrafo 5.3.1, dove il controllo sulla distanza è valutato nello stesso attraversamento; i centri personalizzati, che quella selezione non attraversano, sono filtrati per raggio soltanto nell'elenco dei centri vicini.

5.6.2 Individuazione del centro più vicino

La funzione che porta l'utente al centro più vicino attraversa entrambi gli insiemi, sia quello remoto che quello personale.Funziona tramite una singola lettura dell'intero elenco, e salvando quello alla distanza minore trovato. Di fatto, in questo scenario sarebbe inutile un ordinamento dell'intero insieme, e comporterebbe solamente un costo evitabile.

Sui centri personalizzati l'attraversamento considera soltanto quelli pertinenti alla patologia selezionata.Sui centri remoti il filtro per patologia è implicito nell'insieme stesso, secondo l'asimmetria discussa al paragrafo 4.3.1, e i record privi di coordinate valide vengono scartati.

Individuato il centro, la viewport non viene semplicemente spostata su di esso ma adattata perché comprenda sia la posizione dell'utente sia quella del centro trovato.Questo perchè l'utente che chiede il centro più vicino non vuole solamente vederlo, vuole vedere quanto è lontano, e una viewport centrata sul solo centro perderebbe tale informazione.

La scheda del centro viene aperta poco dopo l'inizio dello spostamento su di esso. Il ritardo è voluto, per dare il tempo alla mappa e all'animazione di

spostarsi sul centro prima di aprirne la scheda.

5.7 Inserimento dei centri personalizzati

L'inserimento di un centro personalizzato parte da un vincolo di interfaccia dichiarato nel requisito RF-26, ossia che le coordinate non si digitano. Chiedere a un utente di scrivere latitudine e longitudine significa chiedergli un dato che potenzialmente non conosce, in un formato che potrebbe sbagliare poco intuitivo, e per un punto che sta guardando sullo schermo; la posizione viene quindi acquisita dalla mappa.

Le vie di acquisizione sono due, e corrispondono a due situazioni d'uso diverse:

1. Tramite pressione prolungata sulla mappa nel punto desiderato, si apre il modulo assumendo quel punto come posizione.
2. Tramite apertura del modulo dall'elenco dei centri personalizzati e dei preferiti, dove non esiste alcun punto indicato e viene quindi assunto il centro della schermata corrente; il modulo mantiene un comando che riporta la posizione sul centro della mappa, così che l'utente possa spostarsi e poi ottenere il dato.

La validazione è organizzata su due livelli, e la separazione non è arbitraria ma imposta da ciò che ciascun livello può esprimere. Il primo livello è quello dei validatori associati ai singoli campi, che il modulo valuta insieme e che riguardano una proprietà del campo in sé: la denominazione, l'indirizzo e la città non possono essere vuoti, il numero di telefono deve rispettare il formato nazionale, l'indirizzo di posta deve essere sintatticamente corretto. Il secondo livello riguarda le condizioni che coinvolgono più campi contemporaneamente, e che un validatore di campo non può formulare: la presenza di almeno un contatto fra telefono e posta, e la selezione di almeno una patologia. Queste sono verificate al momento della conferma, dopo che il primo livello ha dato esito positivo.

La verifica dei campi avviene mentre l'utente scrive e non soltanto alla conferma. La differenza è sostanziale sul piano dell'esperienza: un modulo che segnala tutti

gli errori al termine della compilazione obbliga a ricostruire quale campo abbia prodotto quale messaggio, mentre una segnalazione immediata collega l'errore al campo nel momento in cui vi si sta lavorando. È anche una risposta al requisito RNF-03, che chiede un riscontro visivo immediato per ogni azione.

Il controllo sul numero di telefono, riportato nel listato 4, vale una spiegazione: la sua struttura riflette il formato reale della numerazione italiana e non una convenzione generica.

```
String? _validateItalianPhone(String? value) {
  if (value == null || value.trim().isEmpty) return null;
  var cleaned = value.replaceAll(RegExp(r'[^\d]'), '');
  if (cleaned.startsWith('39') && cleaned.length > 10) {
    cleaned = cleaned.substring(2);
  }
  if (cleaned.length < 6 || cleaned.length > 11) {
    return 'Il numero deve avere tra 6 e 11 cifre';
  }
  if (!cleaned.startsWith('0') && !cleaned.startsWith('3') &&
      !cleaned.startsWith('1') && !cleaned.startsWith('8')) {
    return 'Inserisci un numero italiano valido (es. +39 02
    ↪  12345678)';
  }
  return null;
}
```

Listato 4: Validazione del numero di telefono secondo il formato della numerazione italiana.

Il controllo procede in tre passi. Prima rimuove ogni carattere che non sia una cifra, perché i recapiti vengono scritti nei modi più vari, con spazi, punti, barre e prefissi fra parentesi, e nessuna di quelle forme è sbagliata dal punto di vista dell'utente. Poi riconosce e rimuove il prefisso internazionale, ma soltanto quando la lunghezza residua lo giustifica: la condizione sulla lunghezza evita di scambiare per prefisso internazionale le prime due cifre di un numero di cellulare che cominci per trentanove, che altrimenti verrebbe privato di quel frammento. Infine verifica che la prima cifra sia compatibile con la numerazione nazionale, cioè zero per la rete fissa, tre per la rete mobile, uno e otto per i numeri di servizio e per le numerazioni gratuite o a pagamento.

La figura 5.8 riassume i tre passi, dall'ingresso della stringa grezza fino all'esito, e rende visibile il punto in cui il valore vuoto imbocca un percorso separato che

non applica alcun rifiuto di campo.

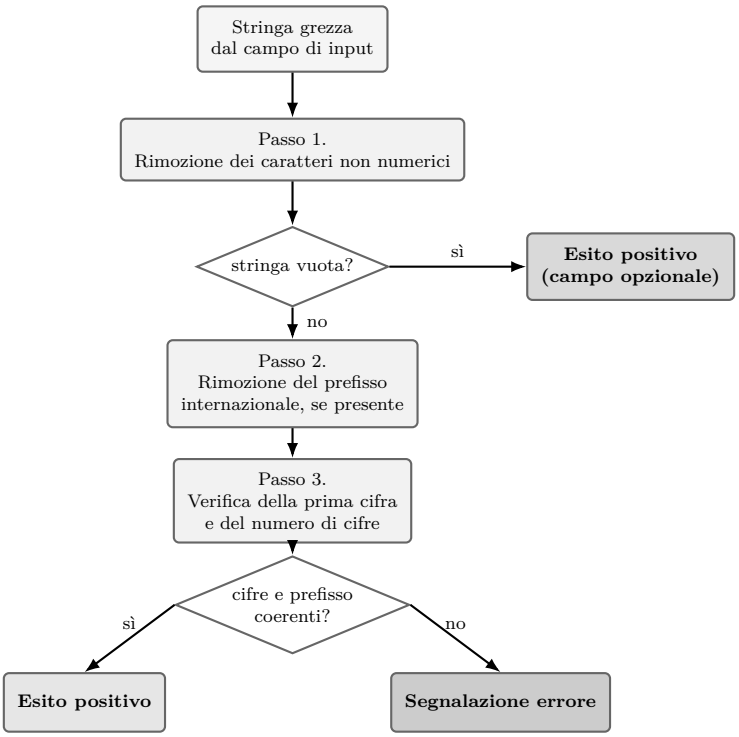


Figura 5.8: Validazione del numero di telefono: i tre passi del controllo e il percorso separato previsto per il valore vuoto.

Un valore vuoto restituisce esito positivo, e non è una dimenticanza: il telefono è opzionale, ed è la verifica di secondo livello a stabilire che almeno un contatto sia presente. Se il validatore di campo rifiutasse il vuoto, un centro con la sola posta elettronica diventerebbe impossibile da salvare.

Ogni centro riceve alla creazione un identificatore ricavato dall'istante di creazione espresso in microsecondi, e quell'identificatore non cambia più. La modifica di un centro esistente non ne genera uno nuovo ma riscrive la voce con lo stesso identificatore, che è la chiave con cui l'aggiornamento distingue la sostituzione dall'inserimento.

5.8 Integrazione con i servizi del dispositivo

Le azioni che seguono la consultazione, cioè raggiungere un centro, telefonargli, scrivergli o condividerne la scheda, non sono realizzate dall'applicazione ma

delegate ai programmi installati sul dispositivo. La delega avviene componendo un collegamento e chiedendo al sistema operativo di aprirlo con l'applicazione competente.

Per le indicazioni stradali sono previste tre destinazioni, riassunte nella tabella 5.5. La scelta di impiegare collegamenti universali di tipo web, invece degli schemi nativi delle singole applicazioni, ha una conseguenza pratica utile: se l'applicazione è installata il sistema la apre direttamente, mentre se non lo è il collegamento viene aperto dal browser, che mostra la stessa mappa. Con uno schema nativo il secondo caso produrrebbe un errore.

Destinazione	**Disponibilità**	**Trattamento del mezzo di trasporto**
Google Maps	entrambe le piattaforme	Automobile, piedi, mezzi pubblici e bicicletta, tutti trasmessi
Waze	entrambe le piattaforme	Non previsto dal collegamento: si avvia con la modalità predefinita
Apple Maps	solo iOS	Automobile, piedi e mezzi pubblici; la bicicletta ricade sull'automobile

Tabella 5.5: Destinazioni di navigazione e trattamento del mezzo di trasporto selezionato.

I due *fallback* riportati nella terza colonna corrispondono agli scenari alternativi del caso d'uso UC-07 e sono stati verificati sui rispettivi collegamenti. Nessuna delle due produce un errore visibile: l'applicazione si apre con la destinazione corretta e una modalità di percorso diversa da quella richiesta, che l'utente può cambiare sul posto. Si è preferito questo comportamento all'alternativa di nascondere la destinazione quando il mezzo selezionato non è trasmissibile, che avrebbe reso incomprensibile la scomparsa di una voce dall'elenco.

Per la chiamata e per la posta il collegamento è composto con gli schemi dedicati, e la richiesta si arresta all'apertura del programma con il recapito già inserito, senza avviare la chiamata: la conferma resta all'utente, com'è giusto che sia per un'azione che ha un costo. I campi dei dati di origine, però, contengono spesso più recapiti nello stesso campo, ed è necessario isolarne uno; viene preso il primo, separando la stringa sui caratteri che nei dati fungono da separatore.

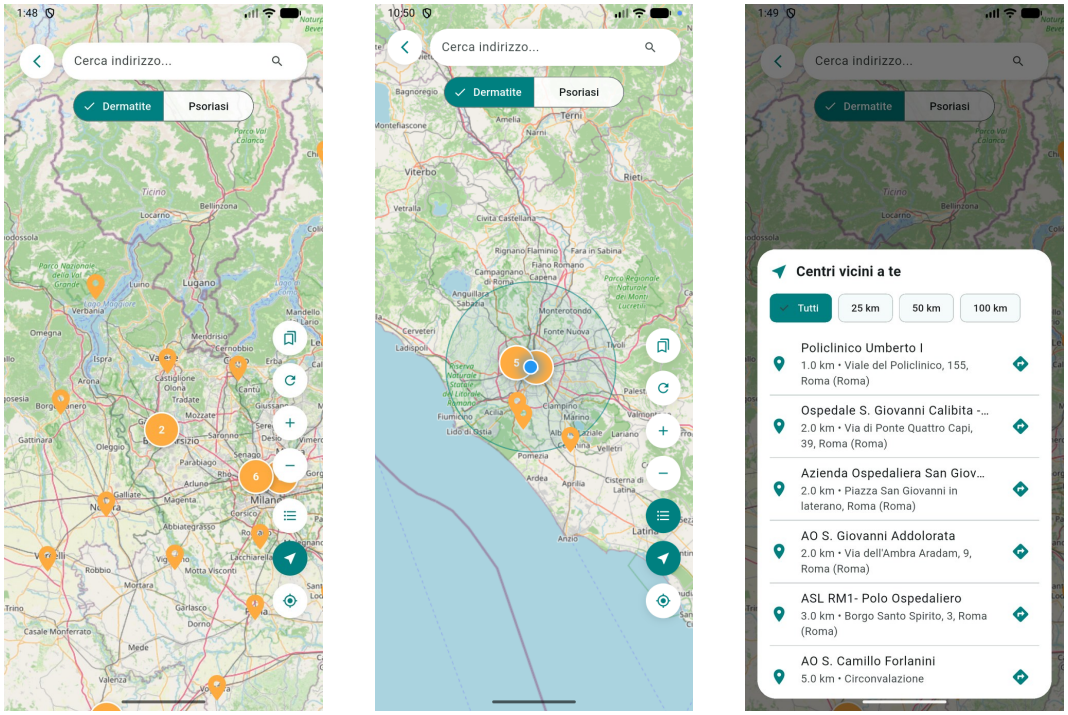
La condivisione compone un testo con la denominazione, l'indirizzo, il recapito telefonico quando presente e un collegamento alla posizione sulla mappa, e lo consegna al pannello di condivisione del sistema. Il collegamento alla posizione

è la parte che rende utile la condivisione: un messaggio con il solo nome della struttura obbliga il destinatario a cercarla, mentre un collegamento apre direttamente il punto sulla mappa.

Tutte le aperture di collegamento sono racchiuse in una gestione degli errori che copre due casi distinti, cioè il fallimento dell'apertura con eccezione e la restituzione di un esito negativo senza eccezione, che si verifica quando nessuna applicazione installata dichiara di saper gestire quello schema. In entrambi i casi l'utente riceve un avviso testuale e la scheda del centro resta aperta, come prevede lo scenario alternativo del caso d'uso UC-07.

5.9 Il modulo in esecuzione

Le figure 5.9 e 5.10 mostrano il modulo in funzione su dispositivo Android, e permettono di verificare visivamente alcune delle scelte descritte nelle pagine precedenti.



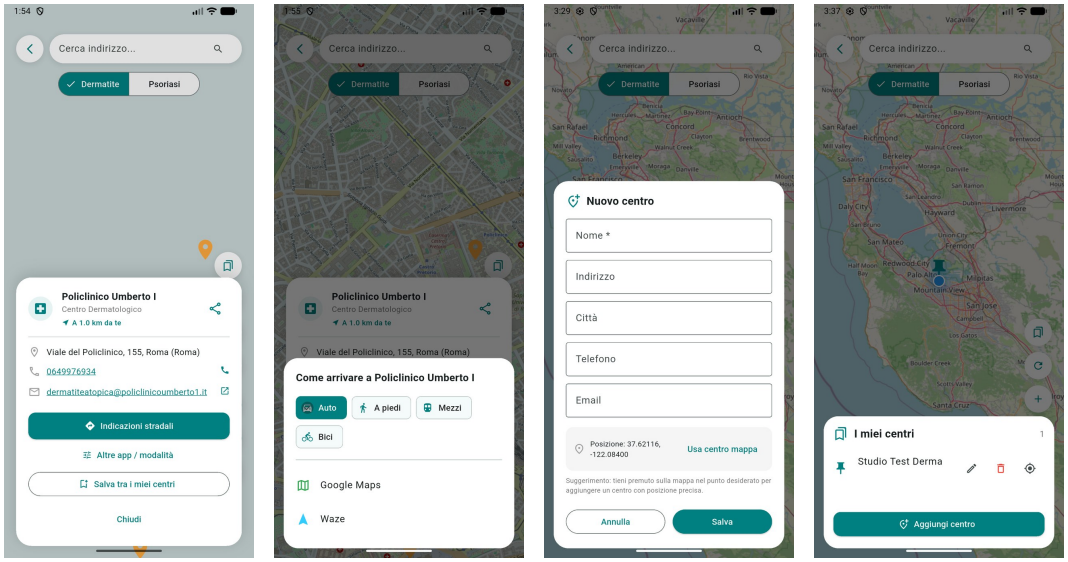
(a) Cluster (b) Filtro per raggio (c) Centri vicini

Figura 5.9: Vista cartografica: cluster numerati, cerchio del filtro per raggio ed elenco ordinato per distanza.

Nella figura 5.9a si riconoscono le bolle numerate prodotte dall'algoritmo del paragrafo 5.3.2, con i marker singoli che convivono con i cluster nella stessa

vista: sono le celle con un solo membro, disegnate come marker ordinari. Si nota anche la colonna dei comandi sul lato destro, nella metà inferiore dello schermo come richiede il requisito RNF-01.

La figura 5.9b mostra il cerchio del filtro attivo, con il riempimento tenue che delimita senza coprire il territorio sottostante. La figura 5.9c riporta l'elenco per prossimità, con le scelte discrete del raggio in testa al pannello e la distanza formattata secondo gli scaglioni descritti al paragrafo 5.6.1.



(a) Scheda (b) Percorso (c) Inserimento (d) Centri propri

Figura 5.10: Pannelli modali: scheda della struttura, scelta del mezzo e dell'applicazione di navigazione, modulo di inserimento ed elenco dei centri personalizzati.

I quattro pannelli della figura 5.10 rendono evidente la scelta discussa al paragrafo 4.2.1: nessuno di essi occupa l'intero schermo, e la mappa resta visibile al di sopra, così che il contesto geografico non venga perso nel momento in cui si consultano i dettagli. Nella figura 5.10b si vedono le scelte del mezzo di trasporto sopra l'elenco delle applicazioni di navigazione, disposizione che rispecchia l'ordine dei passi del caso d'uso UC-07, dove il mezzo viene scelto prima dell'applicazione. La figura 5.10c mostra il modulo di inserimento con il riquadro della posizione acquisita dalla mappa e il comando di riallineamento descritto al paragrafo 5.7.

Capitolo 6

Testing e validazione

6.1 Strategia di testing

I dati arrivano da servizi di terzi, la posizione arriva dal sistema operativo, le azioni finali vengono eseguite da altre applicazioni, e la resa cartografica dipende da immagini scaricate dalla rete. Questi elementi possono rendere la fase di testing potenzialmente molto complessa.

La strategia adottata parte quindi da una domanda, che cos'è verificabile in modo affidabile e ripetibile senza un dispositivo collegato e senza dipendere dalla rete. Sono quindi implementati due tipi di verifica, riassunte nella tabella 6.1.

Livello	**Strumento**	**Elemento verificato**
Unit Testing	Framework di test del linguaggio	Serializzazione dei centri personalizzati, lettura e scrittura dell'archivio locale, calcolo delle distanze geodetiche
Test manuali	Dispositivo ed emulatore	Permessi di posizione, cluster, delega alle applicazioni esterne, comportamento senza rete

Tabella 6.1: Livelli di verifica adottati e rispettivo elemento verificato.

La maggior parte dei test sono anche i più veloci e i più vicini al codice [4, 22]. Gli *unit test* (test unitari) non richiedono alcun ambiente grafico e si eseguono in pochi secondi se non meno; i test manuali sono i più lunghi e sono stati riservati a ciò che i primi non possono coprire.

6.2 Unit test sulla logica applicativa

Gli Unit Test sono dodici, raggruppati in tre insiemi. La tabella 6.2 li elenca con l'esito e con il requisito a cui ciascun insieme si riferisce.

N.	Caso di test	Esito
Serializzazione dei centri personalizzati (RF-29)		
1	La conversione verso JSON e il valore di ritorno conservano tutti i campi	Superato
2	I campi opzionali vuoti o di soli spazi diventano *null*	Superato
3	In assenza dell'elenco delle patologie vengono assegnate entrambe	Superato
4	Coordinate mancanti diventano zero senza sollevare errori	Superato
5	La copia con modifica altera i soli campi richiesti e conserva l'identificatore	Superato
Archivio locale dei centri personalizzati (RF-29, RNF-11)		
6	Un archivio vuoto restituisce un elenco vuoto	Superato
7	Scrittura e successiva lettura conservano i centri e il loro ordine	Superato
8	Un contenuto corrotto non propaga l'errore e restituisce un elenco vuoto	Superato
9	Un documento JSON valido ma di forma diversa da un elenco viene ignorato	Superato
Calcolo geodetico delle distanze (RF-21, RF-22)		
10	La distanza Milano-Roma corrisponde al valore ortodromico noto	Superato
11	La distanza di un punto da se stesso è nulla	Superato
12	La distanza è simmetrica rispetto allo scambio degli estremi	Superato

Tabella 6.2: Prove unitarie sulla logica applicativa: dodici casi, tutti superati.

I casi 2, 3 e 4 verificano comportamenti che l'uso normale non produce mai, ed è per questo che sono stati scritti. Un campo opzionale vuoto, un elenco di patologie assente e una coordinata mancante non si ottengono compilando il modulo di inserimento, che li rifiuterebbe; si ottengono leggendo un archivio scritto da una versione precedente dell'applicazione, oppure un archivio manomesso. Sono percorsi che senza un test automatico resterebbero non esercitati fino al momento in cui, sul dispositivo di un utente, provocherebbero un errore all'avvio.

Lo stesso vale per i casi 8 e 9, che riguardano il requisito RNF-11. Il caso 8 inietta nell'archivio una stringa non interpretabile e verifica che la lettura restituisca un elenco vuoto invece di propagare l'errore. Il caso 9 è la variante meno evidente e più insidiosa: il contenuto è un documento JSON perfettamente valido, ma rappresenta un oggetto e non un elenco. Una lettura che si limitasse a intercettare gli errori di interpretazione supererebbe il caso 8 e fallirebbe il caso 9, perché l'interpretazione riuscirebbe e l'errore comparirebbe dopo, al primo tentativo di scorrere il risultato. La distinzione fra i due casi è il motivo per cui la lettura verifica esplicitamente il tipo del contenuto interpretato.

Il caso 10 richiede una precisazione di metodo. Il valore atteso non è stato ricavato eseguendo il codice e registrandone il risultato, procedimento che avrebbe prodotto un test capace di rilevare soltanto le variazioni future ma non un errore già presente. È stato invece assunto dall'esterno il valore noto della distanza ortodromica fra le due città, circa quattrocentosettantasette chilometri, con una tolleranza di sei chilometri che assorbe le differenze fra i possibili raggi terrestri di riferimento e l'arbitrarietà delle coordinate scelte per rappresentare due città. Un test costruito in questo modo rileva anche l'errore che più facilmente si commette in questo ambito, cioè il calcolo di una distanza euclidea sui gradi: quel calcolo, sulle stesse coordinate, restituirebbe un valore incompatibile con la tolleranza, e il test fallirebbe.

6.3 Verifiche manuali e compatibilità

Le parti del modulo che i test automatici non raggiungono sono state verificate manualmente su emulatore Android e su dispositivo Android fisico. La tabella 6.3 riporta gli scenari percorsi e l'esito osservato.

Scenario	**Comportamento atteso**	**Esito**
Permesso concesso	La mappa viene centrata sulla posizione e l'indicatore la segue negli spostamenti	Conforme
Permesso negato una volta	L'applicazione resta utilizzabile sulla viewport predefinita e la richiesta può essere ripresentata	Conforme
Permesso negato in modo permanente	Compare il collegamento alle impostazioni dell'applicazione	Conforme
Localizzazione disattivata	Compare il collegamento alle impostazioni di sistema, diverso dal precedente	Conforme
Avvio senza rete con cache presente	I marker compaiono dai dati conservati, senza avviso di errore	Conforme
Avvio senza rete e senza cache	Compare l'avviso di errore ed è disponibile il comando di aggiornamento	Conforme
Cluster in area urbana	Le bolle riportano il numero corretto e si scompongono all'avvicinamento	Conforme
Spostamento continuo della viewport	I marker si aggiornano al termine del gesto e non durante	Conforme
Delega a navigatore, telefono e posta	Si apre l'applicazione competente con il dato precompilato	Conforme
Apertura della tastiera nella barra di ricerca	I comandi flottanti si ritraggono e la mappa non si ridimensiona	Conforme

Tabella 6.3: Scenari verificati manualmente su Android e relativo esito.

Sulla compatibilità fra le due piattaforme va dichiarata una limitazione. Non avendo a disposizione una macchina Apple, né un dispositivo iOS, la verifica su quella piattaforma non è stata svolta. Il codice è scritto per entrambe le piattaforme e i punti in cui le distingue sono espliciti, ma il comportamento su iOS resta previsto e non osservato.

Capitolo 7

Conclusioni

7.1 Risultati raggiunti

Il lavoro ha prodotto una mappa funzionante, che raccoglie da elenchi pubblici i centri dermatologici dedicati alla dermatite atopica e alla psoriasi, li mostra su una mappa interattiva con clustering dinamico dei marker, li filtra per patologia e per distanza, li integra con voci inserite a mano e salvate sul dispositivo, e delega ai programmi installati le azioni che seguono la consultazione come, per esempio, la creazione del percorso.

Rispetto alla domanda da cui il lavoro è partito, cioè dove si trovi il centro più vicino che segue una determinata patologia e come raggiungerlo, la risposta è ora ottenibile con due tocchi a partire dall'apertura della schermata, rispetto alla ricerca manuale degli elenchi o di diverse mappe online. Il contributo non sta nei dati, che restano quelli censiti da altri, ma nel modo in cui vengono interrogati: per prossimità, in un unico posto.

Fra i risultati va contato anche il sistema di verifica, con dodici unit test superati sulla serializzazione, persistenza e sul calcolo geodetico.

A livello personale, il lavoro ha avuto un valore che vale menzionare. La scelta iniziale di rinunciare a una piattaforma di mappe commerciale, motivata al paragrafo 4.1.1 da ragioni di costo e di riproducibilità, ha comportato la necessità di costruire a mano funzioni che quella piattaforma avrebbe fornito comprese nel prezzo. Di conseguenza, il modulo dell'algoritmo di clustering del paragrafo 5.3.2 e i ragionamenti annessi, esistono perché quella funzione non era disponibile.

7.2 Criticità e limiti attuali

I limiti che restano sono riconducibili a tre punti.

1. La cache degli elenchi non ha una scadenza, per cui i dati mostrati restano quelli dell'ultimo *download* finché l'utente non richiede esplicitamente un aggiornamento, come discusso al paragrafo 4.3.2.

2. Il modulo dipende dagli elenchi pubblici di terzi, sui quali non si ha controllo ne sul formato e ne sulla loro disponobilità. Le opzioni migliori valutate per rimuovere questi limiti sono tema del paragrafo 7.3.

7.3 Sviluppi futuri

Le opzioni di sviluppo future che sembrano più opportune non sono l'aggiunta di nuove funzioni, ma la rimozione dei limiti individuati nel paragrafo precedente. Le tre che seguono sono ordinate per rapporto di beneficio/costo.

7.3.1 Un livello intermedio di normalizzazione

L'intervento con l'effetto più ampio consiste nell'integrare fra le sorgenti pubbliche e l'applicazione un servizio proprio (come un *middleware* o un'*API*), che interroghi periodicamente gli elenchi originali, ne normalizzi il contenuto una volta ed esponga ai client un unico formato stabile. Questo è un elemento che non è stato ancora sviluppato, per le ragioni mostrate al paragrafo 4.1.

Un servizio di questo genere risolverebbe tre limiti distinti:

1. Renderebbe superflue le catene di alias nel codice, perché l'applicazione riceverebbe record già omogenei e i nomi dei campi resterebbero confinati nel servizio, che è il luogo dove il requisito RNF-17 li voleva.

2. Sul lato del servizio permetterebbe di riconoscere i cambiamenti di formato delle sorgenti, che oggi si manifestano soltanto sul dispositivo dell'utente e sotto forma di elenco vuoto o malformato.

3. Ridurrebbe drasticamente le richieste dirette ai servizi pubblici: le interrogazioni verrebbero effettuate da un solo ente a intervalli regolari invece che da ciascuna copia dell'applicazione.

7.3.2 Sincronizzazione fra dispositivi

I centri inseriti dall'utente vivono ad oggi soltanto sul dispositivo su cui sono stati creati. Cambiando telefono vengono persi, e non esiste modo di consultarli da un secondo dispositivo. Una sincronizzazione remota risolverebbe entrambi i problemi, e sul piano tecnico sarebbe integrabile sull'impianto esistente: i centri personalizzati hanno già una rappresentazione tipizzata, una serializzazione simmetrica e un identificatore stabile, ossia i tre requisiti principali di un sistema tale.

Un aspetto delicato non è la sincronizzazione ma ciò che essa comporta sul piano della riservatezza. L'applicazione, nella versione attuale, non richiede alcuna registrazione e non trasmette all'esterno alcun dato personale, come stabiliscono i requisiti RNF-14 e RNF-15; l'elenco dei centri dermatologici che una persona salva è un'informazione da cui si potrebbe presumere la sua condizione di salute, e trasferirla su un servizio remoto la trasforma in un dato sanitario personale custodito da terzi, con le conseguenze normative che ne derivano. Una sincronizzazione realizzata senza affrontare questo punto peggiorerebbe l'applicazione invece di migliorarla, e la direzione corretta sarebbe una cifratura eseguita sul dispositivo con una chiave che il servizio non possiede, in modo che il servizio conservi dati che non è in grado di leggere nemmeno in caso esso venisse violato.

Un ulteriore possibile miglioramento, di natura diversa, consisterebbe nel permettere all'utente di proporre alle sorgenti originali le strutture che ha salvato e che negli elenchi pubblici non compaiono. Chiuderebbe il ciclo fra i due ruoli: chi consuma l'elenco è anche nella posizione migliore per accorgersi di ciò che manca, e restituirebbe valore ai portali da cui il progetto prende i propri dati; richiederebbe però una moderazione redazionale, che è un problema più organizzativo piuttosto che informatico.

7.3.3 Percorsi e mappe non raster vettoriali

Tutte le distanze mostrate dal modulo sono geodetiche (sono corrette in base alla curvatura terrestre), limitazione discussa al paragrafo 5.6.1. L'introduzione di un servizio di *routing* permetterebbe di ordinare i centri per il tempo di viaggio effettivo invece che per distanza in linea d'aria, grandezza a cui l'utente è realmente interessato. Per esempio, in un contesto montano, o in un'area urbana

congestionata, il centro geograficamente più vicino non è necessariamente quello alla distanza minore in linea d'aria o con la strada più corta (a causa di fattori come il traffico).

Lato grafico, il passaggio dai tile raster ad una mappa vettoriale porterebbe due vantaggi:

1. La nitidezza a qualunque livello di ingrandimento e su qualunque densità di schermo (*DPI*), che le immagini raster non possono garantire.
2. La possibilità di personalizzare lo stile della mappa attenuando gli elementi meno importanti. Per esempio, una mappa in cui le strutture sanitarie vengano evidenziate e il resto del contesto sia reso più tenue metterebbe in risalto l'informazione che l'utente sta cercando.

Un'ultima funzione utile sarebbe il *predownload* in locale dei tile di un'area scelta dall'utente, per consultare la mappa in assenza di rete. Attualmente il salvataggio in locale riguarda soltanto gli elenchi dei centri, come descritto al paragrafo 4.3.2, e in assenza di rete i marker compaiono su uno sfondo vuoto. Quindi i marker restano disponibili, ma il contesto geografico che li rende leggibili viene perso. Su questa strada occorrerebbe procedere con criterio per rispettare la politica d'uso (*ToS*) dei tile pubblici, che vieta espressamente il *download* massivo di aree [27], e che renderebbe quindi necessario l'utilizzo di un provider diverso oppure la predisposizione di un proprio servizio di tile.

Bibliografia

[1] Christopher E. M. Griffiths e Jonathan N. W. N. Barker. «Pathogenesis and clinical features of psoriasis». In: *The Lancet* 370.9583 (2007), pp. 263–271 (cit. alle pp. 1, 5).

[2] Ministero della Salute. *Piano Nazionale della Cronicità*. Roma: Direzione Generale della Programmazione Sanitaria, 2016 (cit. alle pp. 1, 6).

[3] Roger W. Sinnott. «Virtues of the Haversine». In: *Sky and Telescope* 68.2 (1984), p. 159 (cit. a p. 53).

[4] Ian Sommerville. *Software Engineering*. 10th. Pearson, 2015. ISBN: 978-1292096131 (cit. a p. 62).

[5] Stephan Weidinger e Natalija Novak. «Atopic dermatitis». In: *The Lancet* 387.10023 (2016), pp. 1109–1122 (cit. alle pp. 1, 5).

[6] World Health Organization. *Global report on psoriasis*. Ginevra: World Health Organization, 2016. ISBN: 978-92-4-156518-9 (cit. a p. 1).

Sitografia

[7] Antonello Galipò. *latlong2. A Dart library for common latitude and longitude calculations*. 2026. URL: https://pub.dev/packages/latlong2 (visitato il giorno 20/08/2026) (cit. a p. 38).

[8] Apple. *Requesting authorization to use location services*. 2026. URL: https://developer.apple.com/documentation/corelocation/requesting-authorization-to-use-location-services (visitato il giorno 20/08/2026) (cit. a p. 38).

[9] Baseflow. *geolocator. Geolocation plugin for Flutter*. 2026. URL: https://pub.dev/packages/geolocator (visitato il giorno 20/08/2026) (cit. a p. 38).

[10] Dart team. *http. A composable API for making HTTP requests in Dart*. 2026. URL: https://pub.dev/packages/http (visitato il giorno 20/08/2026) (cit. a p. 38).

[11] Dermatopia. *Centri per la dermatite atopica*. 2026. URL: https://centri.dermatopia.it (visitato il giorno 20/08/2026) (cit. alle pp. 1, 6).

[12] Fleaflet. *flutter_map documentation*. 2026. URL: https://docs.fleaflet.dev (visitato il giorno 20/08/2026) (cit. a p. 8).

[13] Fleaflet. *flutter_map. A versatile mapping package for Flutter*. 2026. URL: https://pub.dev/packages/flutter_map (visitato il giorno 20/08/2026) (cit. alle pp. 8, 38).

[14] Flutter Community. *share_plus. Flutter plugin for sharing content*. 2026. URL: https://pub.dev/packages/share_plus (visitato il giorno 20/08/2026) (cit. a p. 38).

[15] Flutter Community. *shared_preferences. Persistent key-value store for Flutter*. 2026. URL: https://pub.dev/packages/shared_preferences (visitato il giorno 20/08/2026) (cit. a p. 38).

[16] Flutter Community. *url_launcher. Flutter plugin for launching a URL*. 2026. URL: https://pub.dev/packages/url_launcher (visitato il giorno 20/08/2026) (cit. a p. 38).

[17] Google. *Dart programming language*. 2026. URL: https://dart.dev (visitato il giorno 20/08/2026) (cit. alle pp. 10, 37).

[18] Google. *Flutter documentation*. 2026. URL: https://docs.flutter.dev (visitato il giorno 20/08/2026) (cit. alle pp. 10, 37).

[19] Google. *Google Maps Platform. Prezzi*. 2026. URL: https://mapsplatform.google.com/pricing/ (visitato il giorno 20/08/2026) (cit. a p. 8).

[20] Google. *Material Design 3*. 2026. URL: https://m3.material.io (visitato il giorno 20/08/2026) (cit. alle pp. 10, 37).

[21] Google. *Request location permissions*. 2026. URL: https://developer.android.com/develop/sensors-and-location/location/permissions (visitato il giorno 20/08/2026) (cit. a p. 38).

[22] Google. *Testing Flutter apps*. 2026. URL: https://docs.flutter.dev/testing/overview (visitato il giorno 20/08/2026) (cit. a p. 62).

[23] Mapbox. *Mapbox Pricing*. 2026. URL: https://www.mapbox.com/pricing (visitato il giorno 20/08/2026) (cit. a p. 8).

[24] Meta Platforms. *React Native*. 2026. URL: https://reactnative.dev (visitato il giorno 20/08/2026) (cit. a p. 10).

[25] OpenStreetMap Foundation. *Copyright and License*. 2026. URL: https://www.openstreetmap.org/copyright (visitato il giorno 20/08/2026) (cit. a p. 8).

[26] OpenStreetMap Foundation. *Nominatim Usage Policy*. 2026. URL: https://operations.osmfoundation.org/policies/nominatim/ (visitato il giorno 20/08/2026) (cit. alle pp. 9, 52).

[27] OpenStreetMap Foundation. *Tile Usage Policy*. 2026. URL: https://operations.osmfoundation.org/policies/tiles/ (visitato il giorno 20/08/2026) (cit. alle pp. 9, 69).

[28] OpenStreetMap Wiki. *Slippy map tilenames*. 2026. URL: https://wiki.openstreetmap.org/wiki/Slippy_map_tilenames (visitato il giorno 20/08/2026) (cit. a p. 9).

[29] Vicini di Pelle. *Centri per la psoriasi e l'artrite psoriasica*. 2026. URL: https://www.vicinidipelle.it (visitato il giorno 20/08/2026) (cit. alle pp. 1, 6).